



CUSTOMER EDUCATION SERVICES

DFT Compiler 1 Workshop

Lab Guide

30-I-011-SLG-011

Version 2007.03

Synopsys Customer Education Services
700 East Middlefield Road
Mountain View, California 94043

Workshop Registration: 1-800-793-3448

www.synopsys.com

Copyright Notice and Proprietary Information

Copyright © 2007 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Synopsys permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any. Licensee must assign sequential numbers to all copies. These copies shall contain the following legend on the cover page:

"This document is duplicated with the permission of Synopsys, Inc., for the exclusive use of _____ and its employees. This is copy number _____."

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AMPS, Cadabra, CATS, CRITIC, CSim, Design Compiler, DesignPower, DesignWare, EPIC, Formality, HSIM, HSPICE, iN-Phase, in-Sync, Leda, MAST, ModelTools, NanoSim, OpenVera, PathMill, Photolynx, Physical Compiler, PrimeTime, SIVL, SNUG, SolvNet, System Compiler, TetraMAX, VCS, and Vera are registered trademarks of Synopsys, Inc.

Trademarks (™)

Active Parasitics, AFGen, Apollo, Astro, Astro-Rail, Astro-Xtalk, Aurora, AvanTestchip, AvanWaves, BOA, BRT, ChipPlanner, Circuit Analysis, Columbia, Columbia-CE, Comet 3D, Cosmos, CosmosEnterprise, CosmosLE, CosmosScope, CosmosSE, Cyclelink, DC Expert, DC Professional, DC Ultra, Design Advisor, Design Analyzer, Design Vision, DesignerHDL, DesignTime, Direct RTL, Direct Silicon Access, Discovery, Dynamic Model Switcher, Dynamic-Macromodeling, EDAnavigator, Encore, Encore PQ, Evaccess, ExpressModel, Formal Model Checker, FoundryModel, Frame Compiler, Galaxy, Gatran, HANEX, HDL Advisor, HDL Compiler, Hercules, Hercules-II, Hierarchical Optimization Technology, High Performance Option, HotPlace, HSIM plus, HSPICE-Link, i-Virtual Stepper, iN-Tandem, Integrator, Interactive Waveform Viewer, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, JVXtreme, Liberty, Libra-Passport, Libra-Visa, Library Compiler, Magellan, Mars, Mars-Rail, Mars-Xtalk, Medici, Metacapture, Milkyway, ModelSource, Module Compiler, Nova-ExploreRTL, Nova-Trans, Nova-VeriLint, Orion_ec, Parasitic View, Passport, Planet, Planet-PL, Planet-RTL, Polaris, Power Compiler, PowerCODE, PowerGate, ProFPGA, ProGen, Prospector, Raphael, Raphael-NES, Saturn, ScanBand, Schematic Compiler, Scirocco, Scirocco-i, Shadow Debugger, Silicon Blueprint, Silicon Early Access, SinglePass-SoC, Smart Extraction, SmartLicense, Software, Source-Level Design, Star-RCXT, Star-SimXT, Taurus, TimeSlice, TimeTracker, Timing Annotator, TopoPlace, TopoRoute, Trace-On-Demand, True-Hspice, TSUPREM-4, TymeWare, VCSEXPRESS, VCSi, VerificationPortal, VFormal, VHDLCompiler, VHDLSystem Simulator, VirSim, and VMC are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in, SVP Café, and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

All other product or company names may be trademarks of their respective owners.

Document Order Number: 30-I-011-SSG-011
DFT Compiler 1 Workshop Lab Guide

1

From Reading to Handing Off

Learning Objectives

After completing this lab, you should be able to:

- Identify the primary steps in running the tool
- Identify the primary command for evaluating tool transformations
- Find out the purpose and syntax of commands and variables



Lab Duration:
30 minutes

Lab 1

Getting Started

If you need help...

```
dc_shell-xg-t> help prev*  
dc_shell-xg-t> printvar *scan*  
dc_shell-xg-t> preview_dft -help  
dc_shell-xg-t> man compile
```

When working interactively, you do not need to type the entire command or option.

```
dc_shell-xg-t> read_test_p -v  
dc_shell-xg-t> all_inputs -cl
```

Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, consult the back portion of this lab for help.

Overview

Unmapped Flow:

1. Read the RTL design into DFTC
2. Create and save a test protocol; verify that the design and the test protocol are compatible
3. Compile and save the gate-level design; exit

Mapped Flow:

4. Read the gate-level design and test protocol into DFTC
5. Specify scan constraints and preview the scan architecture that will result from applying them
6. Insert the scan chain
7. Hand off the design and related files for use by downstream tools; exit

File Locations

All files for this lab except for designs and libraries are located in the directory **lab1_intro**.

Directory Structure

lab1_intro	Current working directory
analyzed	Intermediate acs_read_hdl files
logs	Session log files
unmapped	Unmapped protocol
mapped	Gate-level netlist
mapped_scan	Will contain final netlist
reports	DFTC reports
tmax	Files for downstream tools
scripts	Constraint and run scripts
../ref/rtl/RISC_CORE/vhdl	Design files
../ref/db	Library db files

Lab 1

Instructions

During this lab, you will read the required files into DFTC (Design for Test Compiler) perform DFTC transformations on them and write out the transformed files for use by downstream tools.

Your goal is to execute seven scripts, performing the seven basic steps in running DFTC. The commands in each step will be explained later in the course. For this lab, your goal is to become familiar with the high-level steps in both the Unmapped and Mapped DFT Insertion Flows.

The seven scripts:

Unmapped Flow:

```
1read_design.tcl
2create_test_protocol.tcl
3compile_and_save.tcl
```

Mapped Flow:

```
4read_gates_and_protocol.tcl
5preview_dft.tcl
6insert_dft.tcl
7handoff.tcl
```

Task 1. Execute the Unmapped Flow

1. Invoke DC/DFTC.

```
UNIX% cd lab1_intro
UNIX% dc_shell-xg-t
```

2. Create a shortcut to make running the scripts easier.

```
alias s "source -echo -verbose"
```

3. Read the design into DFTC.

```
s scripts/1read_design.tcl
```

Note: You can use file completion! Type “s scripts/1” then hit the TAB button. DC should show the complete file name now. You can also use the arrow keys to make edits to your command line before executing it.

4. Create and save a test protocol.

```
s scripts/2create_test_protocol.tcl
```

After creating the protocol, the script verifies that the design and the protocol are compatible.

Question 1. What command verifies that the RTL design and the protocol are compatible?

.....

5. Perform a test-ready compile and save the design.

```
s scripts/3compile_and_save.tcl
```

This step is a standard Design Compiler compile command but uses a special option for scan.

Lab 1

Task 2. Execute the Mapped Flow

1. Read the gate-level design and test protocol into DFTC.

```
s scripts/4read_gate_and_protocol.tcl
```

Question 2. What command verifies that the gate-level design and the protocol are compatible?

.....

2. Specify scan constraints and preview the scan architecture that will result.

```
s scripts/5preview_dft.tcl
```

Question 3. How many scan chains does preview_dft report will be inserted?

.....

3. Insert the scan chain(s).

```
s scripts/6insert_dft.tcl
```

Question 4. What command verifies that the inserted scan chain follows design rules?

.....

4. Write out the design and related files to be used by downstream tools.

```
s scripts/7handoff.tcl
```

Task 3. Explore Online 'Quick Reference' Help

You control DFTC using commands and variables.

To display definitions of **commands** use:

- help
- help -v ;# short for help -verbose
- man

To display definitions of **variables** use:

- printvar
- man

Perform the following steps and answer the following questions.

1. Display all the commands available to you by typing 'help'.
2. Use wildcards: display all the **variables** beginning with the string 'test_default'.

Question 5. How many are there?

.....

3. Use the -v option to display 'quick-reference' style syntax.

Question 6. What is the dft_drc option to display estimated fault coverage?

.....

4. Use wildcards to display all variables containing the string "rtldrc".

Question 7. How many are there?

.....

5. Display the detailed information on a command or variable with 'man'.

Question 8. What is the function of the dft_drc command?

.....

.....

Lab 1

Lab Review Questions

Question 9. If you were a new member of a design team or an ASIC design center, which of the seven steps would require you to spend the most time getting information from your team or ASIC customer?

.....

.....

Question 10. What steps require some kind of verification, which, if not satisfactory, requires returning to a previous step?

.....

.....

Question 11. What command appeared most often in the flow, and is used at various steps to evaluate designs and test protocols?

.....

Congratulations! You have completed Lab 1.

Answers / Solutions

Task 1. Execute the Unmapped Flow

You can view the results of running the unmapped flow by looking at:
`./solutions/unmapped_flow.log`

Question 1. What command verifies that the RTL design and the protocol are compatible?

`dft_drc`

Task 2. Execute the Mapped Flow

You can view the results of running the mapped flow by looking at:
`./solutions/mapped_flow.log`

Question 2. What command verifies that the gate-level design and the protocol are compatible?

`dft_drc`

Question 3. How many scan chains does `preview_dft` report will be inserted?

One.

Question 4. What command verifies that the inserted scan chain follows design rules?

`dft_drc`

Question 5. How many are there?

```
printvar test_default*
```

There are 7 variables.

Question 6. What is the `dft_drc` option to display estimated fault coverage?

```
help -v dft_drc
```

The option is `coverage_estimate`

Question 7. How many are there?

```
printvar *rtldrc*
```

There are 2 variables.

Question 8. What is the function of the `dft_drc` command?

```
man dftdrc
```

`dft_drc` checks the current design against test design rules.

Lab Review Questions

Question 9. If you were a new member of a design team or ASIC design center, which of the seven steps would require you to spend the most time getting information from your team or ASIC customer?

Step 2, creating the test protocol, and step 5, defining and confirming the DFT architecture, both require knowledge of your design's testability features, and would probably take the most time.

Question 10. What steps require some kind of verification, which, if not satisfactory, requires returning to a previous step?

```
create_test_protocol, preview_dft,  
insert_dft.
```

Question 11. What command appeared most often in the flow, and is used at various steps to evaluate designs and test protocols?

```
dft_drc.
```


2

Introduction to Design Vision

Learning Objectives

After completing this lab, you should be able to:

- Read a design and related files into Design Vision, the graphical interface to **Design For Test Compiler (DFTC)**
- Compile that design into a test-ready design
- Explore that design in Design Vision
- Save the test-ready design



Lab Duration:
45 minutes

Lab 2

Overview

Commands, Variables, and Files for each objective

This section shows the areas of focus for this lab, and the commands, files, and variables that are taught in each.

1. Set up your environment for reading:
 - `target_library, link_library, search_path, sh_enable_line_editing, alias; .synopsys_dc.setup`
file
2. Read a design into Design Vision:
 - `acs_read_hdl`
3. Compile and Save the Design:
 - `source -e -v; compile -scan, write -out -hierarchy`
4. Explore the Design:
 - `current_design, list_design, report_hierarchy; symbols view, schematics view, zoom`

Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, consult the back portion of this lab for help.

Instructions

Task 1. Set Up Your Environment For Reading

1. Change directories to the project directory `lab2_dv`.

Look at the “dot” files in that directory.

```
UNIX% cd lab2_dv
UNIX% pwd
UNIX% ls -a
```

2. Specify the libraries and append to the search path by editing the `.synopsys_dc.setup` file with a text editor.

Add the following lines to the `.synopsys_dc.setup` file.

```
# for reading and linking
set target_library "sc_max.db"
set link_library  "*" sc_max.db dw_foundation.sldb"
lappend search_path ./scripts
```

The pound (#) at the beginning introduces a TCL comment.

- Question 1.** You use the `set` command to assign values to ‘things’ such as `target_library` and `link_library`; what are these ‘things’ called in TCL?

.....

- Question 2.** When you write out a design to a `.ddc` file, what is the difference in how values defined with the `set` command are treated, compared to how values defined with commands starting with `set_`, such as `set_scan_configuration` or `set_dft_signal`, are treated?

.....

.....

Lab 2

3. Create shortcuts by editing the **.synopsys_dc.setup** file.
 - a. Create the following commonly-used alias for the frequently-used **history** command.
 - b. Then turn on the capability to edit commands in **dc_shell-t**.
 - c. Then save the **.synopsys_dc.setup** file and quit the editor.

```
# shortcuts
history keep 100
alias h "history"
set sh_enable_line_editing true ;# wow - read man page
```

Question 3. To use the history command in the Design Compiler environment (**design_vision/dc_shell-t/etc.**) what would you type at the command line after specifying: **alias h history**?

.....

4. Further examine the **.synopsys_dc.setup** file. Note the two variables **sh_output_log_file** and **sh_command_log_file**. By setting these two variables, you control where a command and an output log file are created. The two files are created using the current time stamp and pid. This sequence of commands can be inserted in most Synopsys tools.
5. Invoke Design Vision from the UNIX prompt in the **lab2_dv** directory:

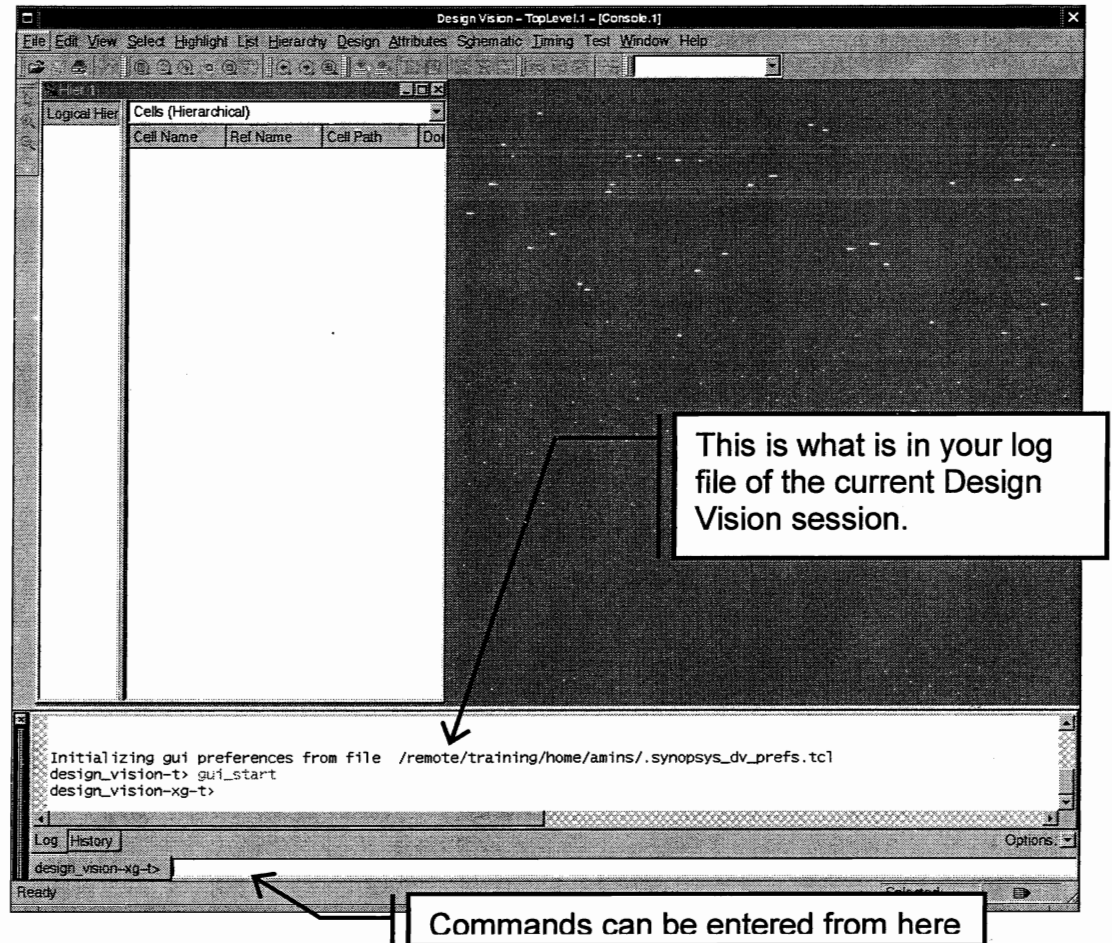
```
UNIX% design_vision
```

Note: You must open Design Vision from the same directory where the **.synopsys_dc.setup** exists, because, as shown in the previous steps, variables such as **search_path** are defined relative to the directory containing the setup file.

Note: If you see any errors while invoking DV, refer to the golden solution at **./solutions/dot.synopsys_dc.setup**

6. List the contents of the **logs** directory. You should see the two log files discussed above.

Task 2. Read a design into Design Vision



Once in Design Vision, you can take advantage of its graphical interface for debugging and troubleshooting problems, or work in `dc_shell-xg-t`, taking advantage of the speed of a shell.

1. Read the file containing the `RISC_CORE` entity or module, and read all associated subfiles, by applying the following command:

Before executing the command, click on the **Log** tab in the lower left corner of the GUI, and make the Session Window as large as possible, so that you can follow the progress of the read.

```
acs_read_hdl -f vhdl \
-hdl_source ../ref/rtl/RISC_CORE/vhdl RISC_CORE
```

Question 4. What does the `acs_read_hdl` command do in addition to reading in the model you specify?

Lab 2

Task 3. Compile and Save the Design

1. **Constrain** the design using the constraints file provided for you in the scripts/ directory called: constraints.tcl.

Design timing and area constraints are needed to direct the next step: compile.

Note: These constraints drive the standard Design Compiler logic synthesis, and are not the focus in this class. Do not spend your lab time trying to learn them.

```
source -e -v constraints.tcl
```

Question 5. The constraint file is in the **scripts** subdirectory; you did not have to specify the directory when you sourced the constraints file, why not?

.....

2. **Compile** the design using the following command:

```
compile -scan
```

The compile command synthesizes the design to the target library by taking the GTECH (Generic Technology) library parts and translating them into the target library parts (in this case, a 0.13 μ training library).

Compile maps to ordinary D flip-flops by default. The `-scan` switch directs DC to translate the GTECH sequential elements directly to special test flip-flops called scan flops. This is called “test-ready compile” or “1-Pass Scan Synthesis”.

3. **Save** the design using the command:

```
write -format ddc -hierarchy \  
      -output mapped/RISC_CORE.ddc
```

This command saves your design in Synopsys internal (DDC) format into the mapped subdirectory. The name of the file is RISC_CORE.ddc. Saving all the sub blocks in the design is enabled by the `-hierarchy` switch.

Task 4. Explore the Design

When inserting scan into your design, problems may occur. Being able to explore your design graphically is a powerful debugging aid.

1. Resize your Session Window, making it smaller so that you can focus on the graphical display at the top part of the GUI.

The following figure shows some key points for navigating in Design Vision.

The screenshot shows the Design Vision interface with the following components and annotations:

- Design Vision - Top Level:1 - [Hier.1] -- RISC_CORE**: The main window title bar.
- Logical Hierarchy**: A tree view on the left showing the design structure. It includes blocks like I_ALU, I_CONTROL, I_DIV_CLK, I_DATA_PATH, I_INSTRN_LAT, I_PRGRM_CNT_TOP, I_REG_FILE, I_RST, and I_STACK_TOP. An arrow points to the **I_STACK_TOP** block.
- Cells (Hierarchical)**: A table on the right showing the current design's components.

Cell Name	Ref Name	Cell Path
I_ALU	ALU	I_ALU
I_CONTROL	CONTROL	I_CONTROL
I_DIV_CLK	DIV_CLK	I_DIV_CLK
I_DATA_PATH	DATA_PATH	I_DATA_PATH
I_INSTRN_LAT	INSTRN_LAT	I_INSTRN_LAT
I_PRGRM_CNT_TOP	PRGRM_CNT_TOP	I_PRGRM_CNT_TOP
I_REG_FILE	REG_FILE	I_REG_FILE
I_RST	RST	I_RST
- Current Design. In the case, RISC_CORE is current design.**: An annotation pointing to the **RISC_CORE** block in the Logical Hierarchy.
- Design Overview Window. Once the designs have been read into Design Vision, this window shows the top-level block and all of the subblock names. A + sign indicates subblocks exist beneath. For instance, select the + sign next to I_STACK_TOP to see the subblocks of the STACK_TOP block**: An annotation pointing to the **I_STACK_TOP** block in the Logical Hierarchy.
- Transcript Window: Here it shows the results of the link command. Many command return 1 and 0 to indicate success and failure, respectively. Be aware that other commands return collections or other values.**: An annotation pointing to the Transcript Window at the bottom.
- Command history log**: An annotation pointing to the Command History log in the Transcript Window.
- Use the Options pulldown to search the log for Errors, Warnings, Informationals, or commands. You can also use F3.**: An annotation pointing to the Options pulldown menu in the Transcript Window.

Question 6. What is the current design?

.....

Lab 2

2. Display the current design from the TCL shell.

You can also display the current design from the TCL shell by using the `current_design` command. Try it:

```
current_design
```

3. Display all the designs from the TCL shell by using the `list_designs` command.

```
list_designs
```

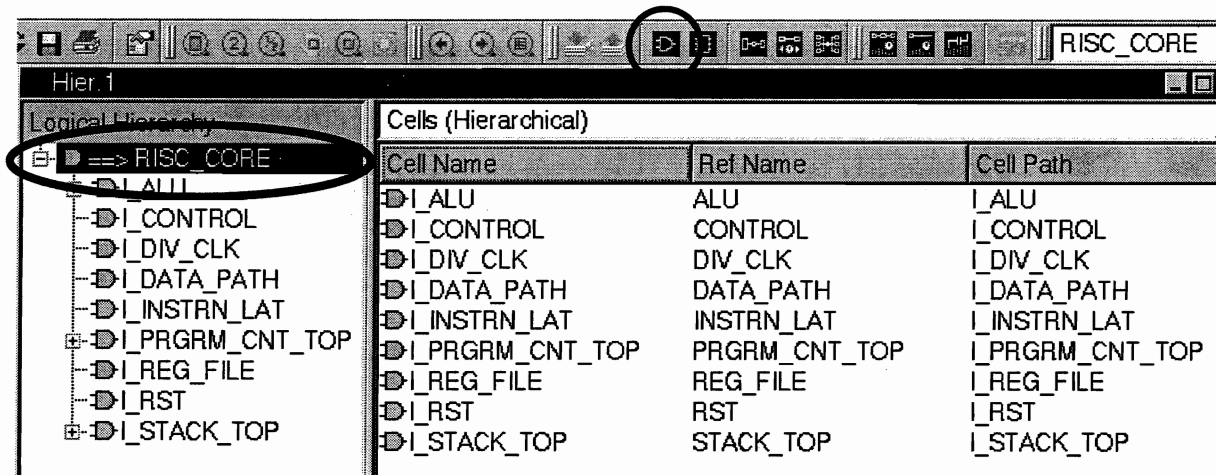
4. Display all the designs graphically.

Select the pull down arrow to the right of the current design; the results match those from the `list_designs` command.

5. Display a schematic of the entire RISC_CORE ASIC.

Specify which part of the ASIC you want to see.

In this case, select RISC_CORE from the Design Overview Window. Notice a list of the subdesigns below RISC_CORE is displayed



Notice the Cell name and ref name of each subdesign are shown. **Cells** are also called **instances**. **Ref names** are also called **designs**.

Lab 2

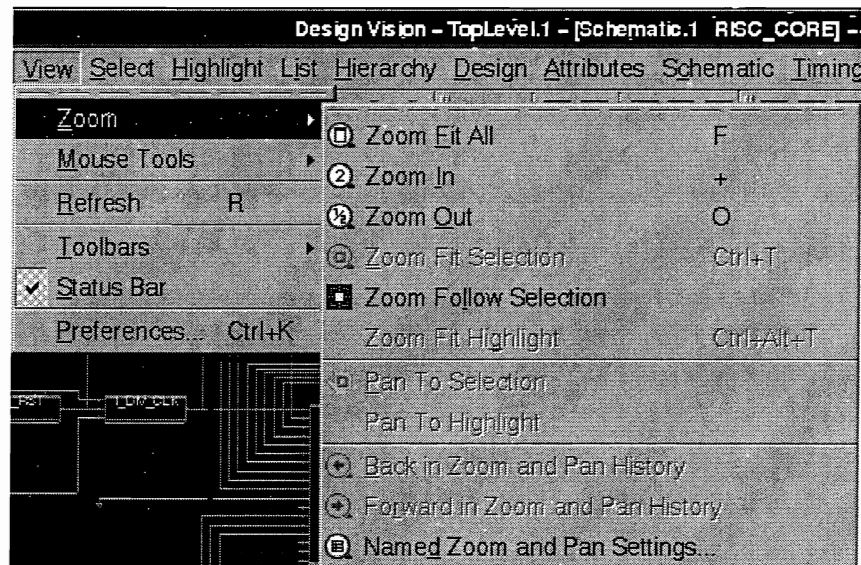
6. Display a schematic of the RISC_CORE ASIC.

Select the orange AND gate in the toolbar. You should see a schematic of the RISC_CORE ASIC.

You can zoom in and out to see things more clearly.

7. Zoom in to examine the schematic in more detail.

Select the **view** pull-down menu to zoom in and out



Note: You may first have to select an area of the schematic which has nothing in it with the mouse to be able to use the Zoom features.

Once in Zoom In mode, you may need to click on the Selection Tool before changing to another mode, such as Zoom Out. The escape key (ESC) is the shortcut for the Selection Tool.

You can also use the middle mouse button to zoom and pan, using so called strokes, e.g. depress the middle mouse button then move the mouse from SW to NE. You should have zoomed in.

8. Display the symbol view (input and output ports) of the RISC_CORE ASIC.

Select the green IC chip icon next to the orange AND gate.



Lab 2

9. Verify that your entire design has been mapped to gates.

The **report_hierarchy** command shows you the entire hierarchy of the design in a textual format, as well as the cells in the design and libraries to which they belong. In this case, you are using the **tsmc** library. If any cells belong to the **gtech** library, you can conclude that there are unmapped cells in your design.

Run the command:

```
report_hierarchy
```

10. Compare the previous result to an unmapped design.

Remove all of the designs, bring in the unmapped design once more, and run the **report_hierarchy** command:

```
remove_design -design  
acs_read_hdl -f vhdl \  
-hdl_source ../ref/rtl/RISC_CORE/vhdl RISC_CORE  
report_hierarchy
```

Notice the RPT-7 message and that all the cells are GTECH library cells because the design is unmapped.

11. Verify that all the library variables have been defined correctly.

```
printvar *_library
```

12. Exit Design Vision by typing:

```
exit
```

Lab Review Questions

Question 7. How do you optimize, map, and scan-replace the flip-flops in a design with **Design Vision**?

.....

Question 8. What is the difference between **Design Vision** and **dc_shell-t**?

.....

Question 9. What is the purpose of the **.synopsys_dc.setup** file?

.....

Question 10. How do you verify the library variables are set up correctly?

.....

Question 11. How do you read VHDL or Verilog code into Design Vision?

.....

Question 12. What is the function of the **target_library** variable?

.....

Congratulations! You have completed Lab 2.

Answers / Solutions

Question 1. You use the `set` command to assign values to ‘things’ such as `target_library` and `link_library`. What are these ‘things’ called in TCL?

Variables

Question 2. When you write out a design to a `.ddc` file, what is the difference in how values defined with the `set` command are treated, compared to how values defined with commands starting with `set_`, such as `set_scan_configuration` or `set_dft_signal`, are treated?

When a variable is applied to a design, it does not “stick” to the design when the design is saved in a `.ddc` format. This is different from a “`set_`” command that can be applied to a `.ddc` file and is saved with the `.ddc` file. A variable must be reapplied each time the design is brought back into Design Compiler.

Question 3. To use the history command in the Design Compiler environment (`design_vision/dc_shell-t/etc.`) what would you type at the command line after specifying: `alias h history`?

`h`

Question 4. What does the `acs_read_hdl` command do in addition to reading in the model you specify?

It finds the files containing all the designs and subdesigns, reads them in, and links them, setting the `current_design` to the top-level design.

Question 5. The constraint file is in the `scripts` subdirectory; you did not have to specify the directory when you sourced the constraints file, why not?

You appended the `scripts` directory to the `search_path` variable earlier in this lab. You can verify this by using “`printvar *path`”

Question 6. What is the current design?

The current design is: `RISC_CORE` (top-level of the ASIC)

Question 7. How do you optimize, map, and scan-replace the flip-flops in a design with **Design Vision**?

Use the **compile -scan** command

Question 8. What is the difference between **Design Vision** and **dc_shell-xg-t**?

Design Vision is the GUI to the Synopsys logic synthesis tools; it is built on **dc_shell** and **dc_shell-t**, which are the command line interfaces.

Question 9. What is the purpose of a **.synopsys_dc.setup** file?

The **.synopsys_dc.setup** file is a script file, which is executed upon invocation of either Design Vision or **dc_shell-t**. It is a useful file for commands that are to be executed each time DC is invoked. For example, it is useful to initialize library variables in this file.

Question 10. How do you verify the library variables are set up correctly?

acs_read_hdl does an automatic link, which produces warnings or errors if the variables are not set correctly.

Question 11. How do you read VHDL or Verilog code into Design Compiler?

To read individual files in, use the **read_vhdl** or **read_verilog** command. To read files that have subblocks, use the **acs_read_hdl** command

Question 12. What is the function of the **target_library** variable?

The **target_library** variable defines the technology library used during compile mapping phase.

Command Sequence For This Lab

```
acs_read_hdl -f vhdl \
    -hdl_source ../ref/rtl/RISC_CORE/vhdl RISC_CORE
source -e -v constraints.tcl
compile -scan
write -format ddc -hier -output mapped/RISC_CORE.ddc
exit
```

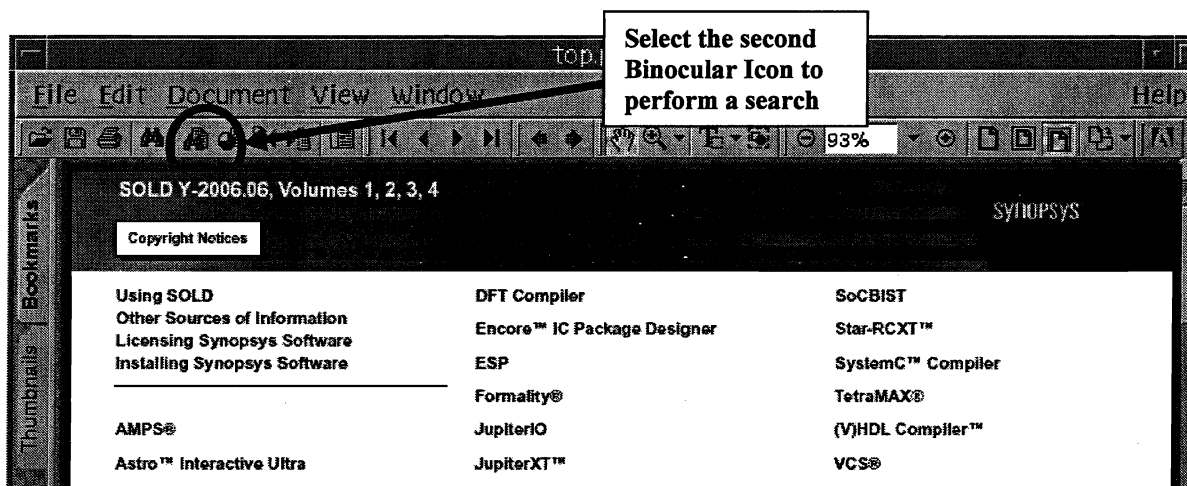
Appendix: Searching Synopsys Online Documentation using Indexes

1. Synopsys stores its product documentation in a collection called **Synopsys Online Documentation (SOLD)**. In addition to the command man pages, **SOLD** is a good place for you to look for more information about Synopsys tools. To access SOLD type **sold** from the UNIX command line followed by an ampersand (&).

```
UNIX> sold &
```

sold is a UNIX alias which invokes **Adobe Acrobat reader** and opens up a file called **top.pdf**, which is the top-level view of all documents.

2. To search for information in SOLD and look only in specific documents, you need to **set up indexes** to customize the searches. SOLD consists of a host of documentation for the various Synopsys tools. Each set of documentation is accessed through its **corresponding index**. For instance, there is an index for the Man Pages and one for **Test Automation**, as well as many others. When the indexes are set up properly, you can use SOLD to **search in one or many indexes** to control how much information you will receive from any search you perform.
3. For information on how to setup indexes consult this SolvNet article <https://solvnet.synopsys.com/retrieve/903898.html> "Script for building SOLD search indexes".
4. To search for information in a SOLD index, first select the binoculars icon (the binoculars icon with documents behind it is the Search button).



3

Creating Test Protocols

Learning Objectives

After completing this lab, you should be able to:

- Provide enough information about the test features of your design that DFTC can create a test protocol.
- Verify that you have told DFTC what it needs to know
- Verify that your protocol and design are compatible
- Debug dft_drc violations using the Design Vision debugger
- Verify that the design has been compiled to the correct library and that all flip-flops have been replaced with scan flip-flops



Lab Duration:
60 minutes

Lab 3

Overview

All files for this lab are located in the directory **lab3_protocol**.

Directory Structure

lab3_protocol	Current working directory
analyzed	Intermediate acs_read_hdl files
logs	Session log files
unmapped	Unmapped protocol
mapped	Gate-level netlist
reports	DFTC reports
tmax	Files for downstream tools
scripts	Constraint and run scripts
../ref/rtl/RISC_CORE/vhdl	Design files
../ref/db	Library files

Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, consult the back portion of this lab for help.

Instructions

During this lab, you will use the Design Vision GUI for debugging dft_drc problems related to test protocols, write a script to create and verify a test protocol and explore a dc_shell-t log file.

Task 1. Debug DRC Violations

1. Change directories to the project directory `lab3_protocol`.
2. Bring up DFTC in shell mode.

```
unix> design_vision
```

Note: Log files will be created in the logs directory.

3. Read in the RISC_CORE design. Type the following in the shell:

```
source scripts/lread_design.tcl
```

4. Run a design rule check.

```
dft_drc
```

Question 1. What error message do you get?

..... *No model found on design*

5. Resolve DFTC's complaint; that is, create a test protocol.

```
create_test_protocol
```

6. Run a design rule check again.

```
dft_drc
```

Question 2. Is the design free of DFT problems or does the report contain many violations?

..... *Yes* *Total 405 pre-DFT VIOLATION*

Lab 3

Task 2. Debug Violations using the GUI

1. Look at the GUI. You should see a Violation Browser showing the same errors shown by the command output of `dft_drc`. You can bring up the violation browser any time using the menu **Test → Browse Violations**.
2. Click on the “+” in front of “PRE-DFT”, then click on the D1 category.
Select the first violation on the right. On the bottom, you will see an explanation of the error.
3. Right click on the first D1 violation, then select “Inspect Violation”, or click on the “Inspect Violation” button on the lower right. A schematic should appear.

Question 3. What logic value is present on the clock?

.....

Question 4. What DFTC command is needed to specify this test clock?

.....

.....

4. Type the command you just noted down (check the back for help).
5. Recreate the test protocol, then switch back to the Violation Browser, and click on “Run DFT DRC”. The schematic window should disappear.

Question 5. Was the number of violations affected at all?

.....

6. Once again, “Inspect Violation” for the first D1 violation shown.

Question 6. What value is shown for TEST_MODE?

.....

Question 7. What DFTC command is needed to specify this test mode?

.....

.....

7. Type the command you just noted down (check the back for help).
8. Recreate the test protocol, then switch back to the Violation Browser, and click on “Run DFT DRC”.

Question 8. Was the number of violations affected at all?

.....

9. Analyze the first D3 violation.

Question 9. What value is shown for Reset?

.....

Question 10. What DFTC command is needed to specify this asynchronous reset (the reset is active low)?

.....

.....

10. Type the command you just noted down (check the back for help).
11. Recreate the test protocol, then switch back to the Violation Browser, and click on “Run DFT DRC”.

Question 11. Was the number of violations affected at all?

.....

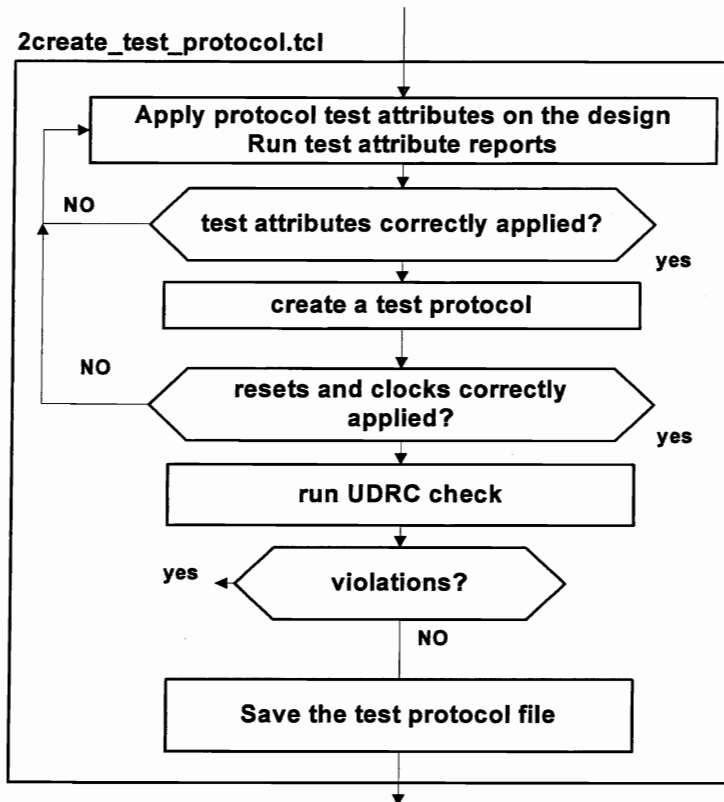
12. Quit the Design Compiler shell.

Lab 3

Task 3. Create a test protocol with a script

Utilize the flowchart below and the protocol test attributes specification to its right as a guide to write and verify a DFTC script to create a test protocol.

STEP 2 OF THE UNMAPPED FLOW



Protocol Test Attributes	
Test Clock Port	Clk
Test Clock Active Time	45 ns
Test Clock Inactive Time	55 ns
Test Mode Port	TEST_MODE
Reset port	Asynchronous active low
	Reset

1. Create a script called `scripts/2create_test_protocol.tcl`.

The tasks are defined in the flow chart above; the values for the attributes in the chart beside it. Your task is to write the DFTC commands to perform the tasks. For this lab, set the values for the test attributes in the script.

Note: You may use the log file to copy and paste the commands you entered in the previous task

2. Execute your script (2create_test_protocol.tcl), as well as the scripts for step 1 and step 3, by running the **run_unmapped.tcl** script in batch mode.

Create a log file for the run:

```
unix% dc_shell-t -f scripts/run_unmapped.tcl | \
tee logs/run_unmapped.log
```

3. Determine if there were any problems in the unmapped flow by looking at **logs/run_unmapped.log**.

Question 12. What command is used to see which DFT signals have been applied to the design?

.....

- Question 13.**
- a. What value is the TEST_MODE port set to?
 - b. What is the name of the test clock port?
 - c. What is the period of the test clock?
 - d. When does the test clock go high?
 - e. When does the test clock go low?
 - f. What is the name of the reset port?
 - g. What type of reset has been assumed?

.....

.....

.....

.....

.....

.....

Question 14. Look at the results of running the **dft_drc** command, are there any errors or warnings?

.....

Lab 3

Task 4. Verify that the Design was Compiled Properly

Question 15. What command tells whether the entire design was compiled to the target library?

.....

1. Verify that the entire design was compiled to the target library.

Question 16. What target library was it compiled to?

.....

Question 17. Now that you know the design has been properly compiled, how do you determine if the flip-flops have been scan replaced?

.....

2. Determine if the design has been properly scan replaced.
3. Exit from DFTC.

Congratulations! You have completed Lab 3.

Answers / Solutions

Task 1. Debug DRC Problems

Question 1. What error message do you get?

Error: No model found on design
0

Question 2. Is the design free of DFT problems or does the report contain many violations?

405 PRE-DFT VIOLATIONS

313 Uncontrollable clock input of flip-flop violations (D1)

92 DFF set/reset line not controlled violations (D3)

Task 2. Debug Violations using the GUI

Question 3. What logic value is present on the clock?

X meaning unknown or unspecified in this case.

Question 4. What DFTC command is needed to specify this test clock?

```
set_dft_signal -view exist -type ScanClock \  
-timing {45 55} -port Clk
```

Question 5. Was the number of violations affected at all?

Yes, the number of D1 violations goes down to 310.

Question 6. What value is shown for TEST_MODE?

X meaning unknown or unspecified in this case.

Question 7. What DFTC command is needed to specify this test mode?

```
set_dft_signal -view exist -type Constant \  
              -active 1 -port TEST_MODE
```

Question 8. Was the number of violations affected at all?

All D1 violations are gone now. 92 D3 violations remain.

Question 9. What value is shown for Reset?

X meaning unknown or unspecified in this case.

Question 10. What DFTC command is needed to specify this asynchronous reset?

```
set_dft_signal -view exist -type Reset \  
              -active 0 -port Reset
```

Question 11. Was the number of violations affected at all?

All violations should be gone now.

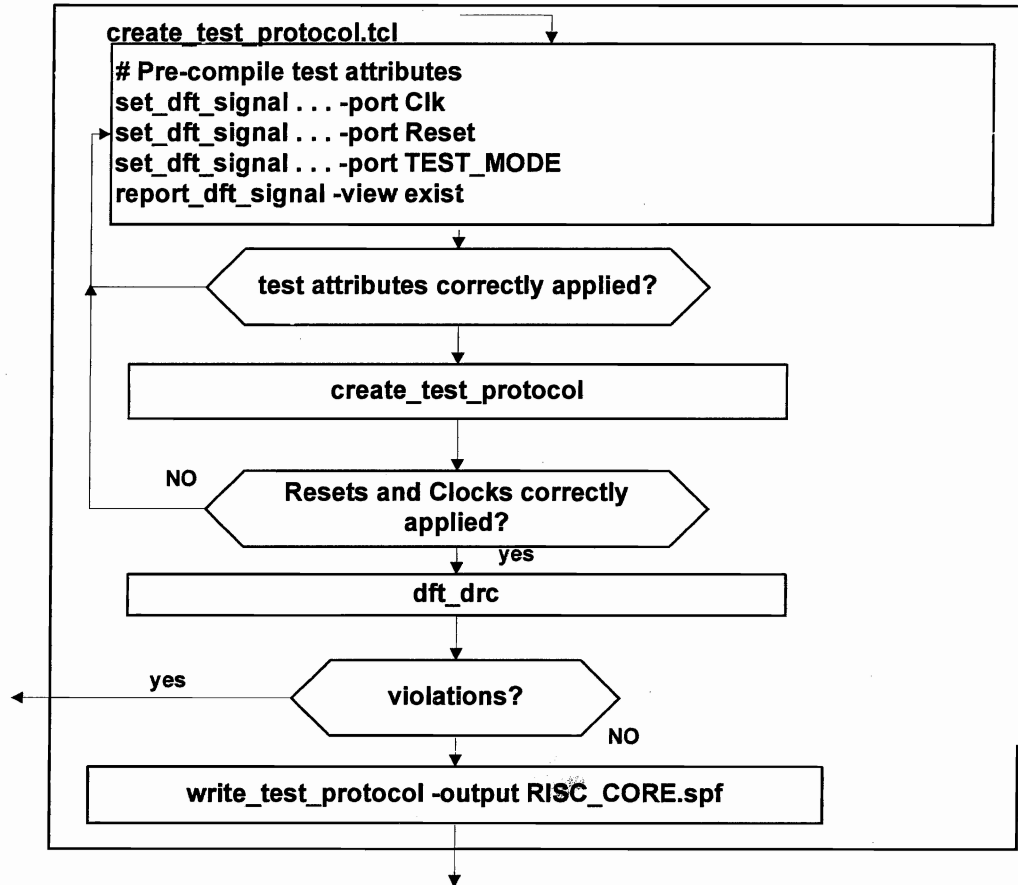
Task 3. Create a Test Protocol with a script

1. Create the script called **2create_test_protocol.tcl**, which is the second step in the **run_unmapped.tcl** script which you learned about in lab1 (create this file in the scripts folder).

If you run into problems, consult **.solutions/2create_test_protocol.tcl**

This flowchart shows you the command flow of the create_test_protocol.tcl file as well as the decision points.

CREATE_TEST_PROTOCOL PORTION OF UNMAPPED FLOW



Question 12. What command is used to see which DFT signals have been applied to the design?

report_dft_signal

Question 13.

- What value is the TEST_MODE port set to?
- What is the name of the test clock port?
- What is the period of the test clock?
- When does the test clock go high?
- When does the test clock go low?
- What is the name of the reset port?
- What type of reset has been assumed?

a. TEST_MODE = 1

b. Test clock port name is: Clk

c. Period of the test clock is: 100 ns

d. Test clock high: 45 ns

Lab 3

Answers / Solutions

e. Test clock low: 55 ns

f. Reset port name is: Reset

g. Reset type is: Asynchronous, active low

Question 14. Looking at the results of running the `dft_drc` command, are there any errors or warnings?

No

Task 4. Verify that the Design was Compiled Properly

Question 15. What command tells whether the entire design was compiled to the target library?

Use the `report_hierarchy` command. All cells should be `cb13fs120_tsmc_max` cells (target library).

Question 16. What target library was it compiled to?

Run the `report_hierarchy` command from the command line. All cells were `cb13fs120_tsmc_max` cells, so the design was properly compiled to the target library

Question 17. Now that you know the design has been properly compiled, how do you determine if the flip-flops have been scan replaced?

Run `report_scan_state` after compile -scan has occurred. Design was properly scan replaced:

```
dc_shell-xg-t> report_scan_state
```

```
*****
```

```
Report : test
```

```
      -state
```

```
Design : RISC_CORE
```

```
*****
```

```
Scan state          : scan cells replaced with loops.
```

4

DFT for Clocks and Resets

Learning Objectives

After completing this lab, you should be able to:

- Debug dft_drc problems with clocks and resets
- Propose a manual solution that could be implemented by you or by another designer
- Use AutoFix to implement a fix



Lab Duration:
45 minutes

Lab 4

Overview

All files for this lab except for designs and libraries are located in the directory **lab4_clocks**.

Directory Structure

lab4_clocks	Current working directory
analyzed	Intermediate acs_read_hdl files
logs	Session log files
unmapped	Unmapped protocol
mapped	Gate-level netlist
reports	DFTC reports
tmax	Files for downstream tools
scripts	Constraint and run scripts
../ref/rtl/RISC_CORE_nodft/vhdl	Design files
../ref/db	Library files
../.solutions	Solution files

Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, consult the back portion of this lab for help.

Instructions

During this lab you will debug Uncontrollable Clock and Reset DFT problems using the Design Vision GUI, propose a fix and implement a fix using AutoFix.

Task 1. Read in Mapped Design

1. Change directories to the project directory `lab4_clocks`.
2. Invoke DC in the GUI mode.

```
UNIX% design_vision
```

3. Read in the mapped design. Remember file name completion!

```
source scripts/4read_gate_and_protocol.tcl
```

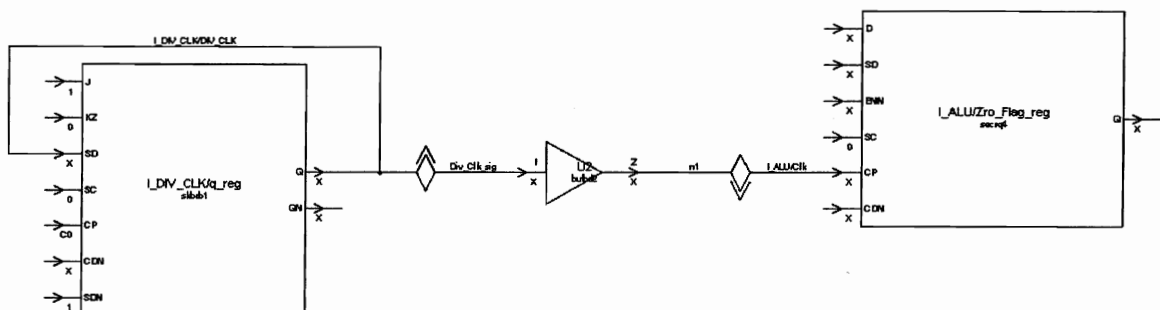
Task 2. Debug Uncontrollable Clocks and Resets

1. From information in the log window, answer the following:

Question 1. How many violations are there?
What type of violations are they?

.....
.....
.....

2. In the Violation Browser, analyze the first D1 violation and you should observe a schematic similar to the one shown below.



Lab 4

Question 2. What is the netlist name for the flip-flop on the left, the cause for the D1 violation?

.....

Question 3. Draw a schematic “fix” for this uncontrollable clock problem.

3. Analyze some of the other D1 violations. Select the Violation Browser tab, then select some of the other D1 violations and examine the schematic.

Question 4. What can you conclude about the D1 violations in the RISC_CORE design?

.....

4. Analyze one of the D3 violations in the same manner as you did the D1 violations.

To expand the second X input to I_RST/q2_reg, double click on the D input.

Question 5. Based on your analysis draw a schematic “fix” for the uncontrollable reset problem.

5. Select some of the other D3 violations and analyze them.

Question 6. What can you conclude about the D3 violations in the RISC_CORE design?

.....

Lab 4

Task 3. Autofix The D1 and D3 violations

You have now analyzed both the uncontrollable clock and uncontrollable reset DFT violations in the RISC_CORE design. Your next task is to fix the violations. As you learned in lecture there are three choices.

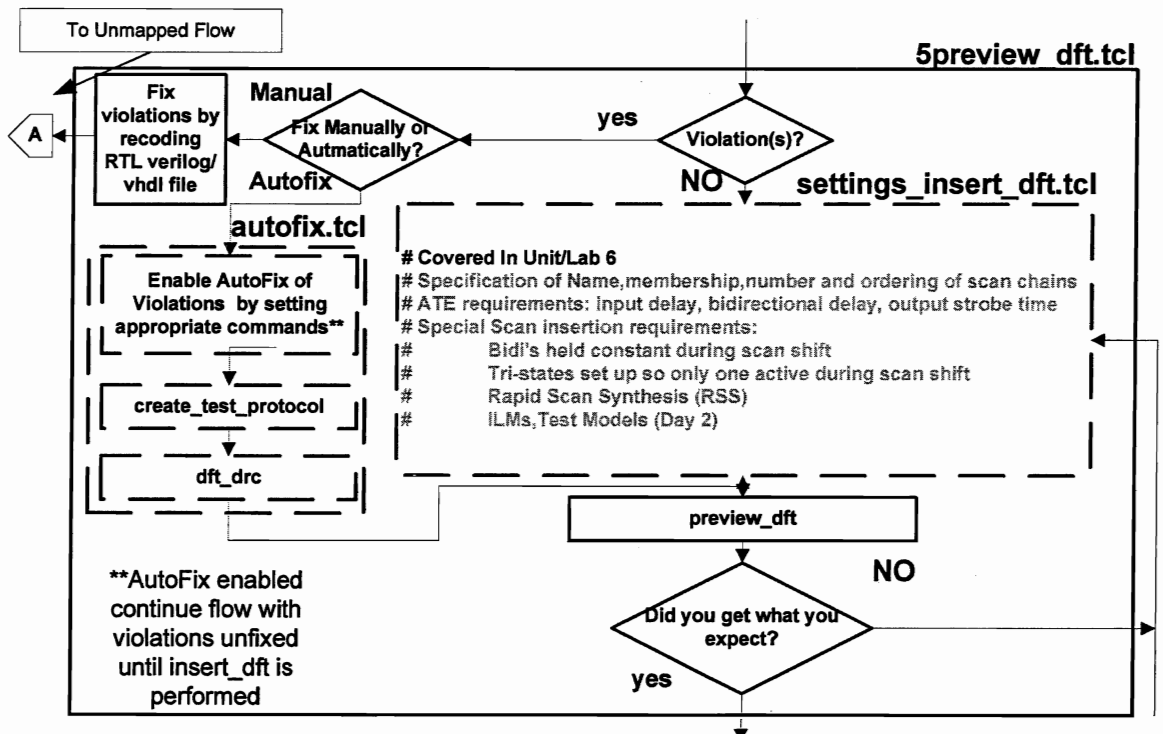
- 1) You could re-code the RTL design and check the design once again using the unmapped DFTflow. When you have no more violations you can continue on to the mapped flow.
- 2) You could draw the solution and have someone else re-code the design for you.
- 3) You can use the AutoFix capability of DFTC.

Question 7. What kind of DFT problems can Autofix solve?

.....
.....

1. Prepare to run Autofix for clocks and resets by modifying the file `scripts/5preview_dft.tcl`

The flowchart below details the AutoFix section of the **mapped** flow. Notice that you need to modify the `5preview_dft.tcl` file to **source** the `autofix.tcl` script **after** sourcing the `settings_insert_dft.tcl` file.



Compare with the flowchart in the Answers / Solutions section.

2. Set up to run AutoFix by creating a script called **scripts/autofix.tcl**.

Utilize the lecture material and the man pages to create a script called **autofix.tcl**, and place it in the **scripts/** directory.

Here is a list of commands related to AutoFix to get you started:

- **set_dft_configuration**
- **set_autofix_configuration**
- **set_dft_signal**

3. Preview what AutoFix will add by running step 5 of the Mapped Flow.

```
source -e -v scripts/5preview_dft.tcl
```

4. Evaluate the previewed AutoFix results.

Take a careful look at the results of running the **preview_dft** command. Notice the sections of the report titled **Test Point Plan Report**, and **DFT signals**. These sections relate directly to the AutoFix portion of the flow.

Compare your results carefully with those in the Answers / Solutions section—it is easy to make a mistake and not realize it.

Question 8. How many AutoFix test points are there? Why?

.....

5. Perform both AutoFix and scan insertion by running step 6 of the DFT flow.

```
source -e -v scripts/6insert_dft.tcl
```

6. Look at the results of running **dft_drc -coverage_estimate**.

Question 9. What were the number of violations?
Were there any cells with violations?
What was the fault coverage?

.....

.....

.....

Lab 4

Question 10. What kind of fault coverage would you expect to get, if you had not AutoFixed the design?

.....

7. Try it and compare coverages.
8. Exit DC.

Congratulations! You have completed Lab 4.

Answers / Solutions

Task 2. Debug Uncontrollable Clocks and Resets

Question 1. How many violations are there? What type of violations are they?

398 PRE-DFT VIOLATIONS

309 Uncontrollable clock input of flip-flop violations (D1)

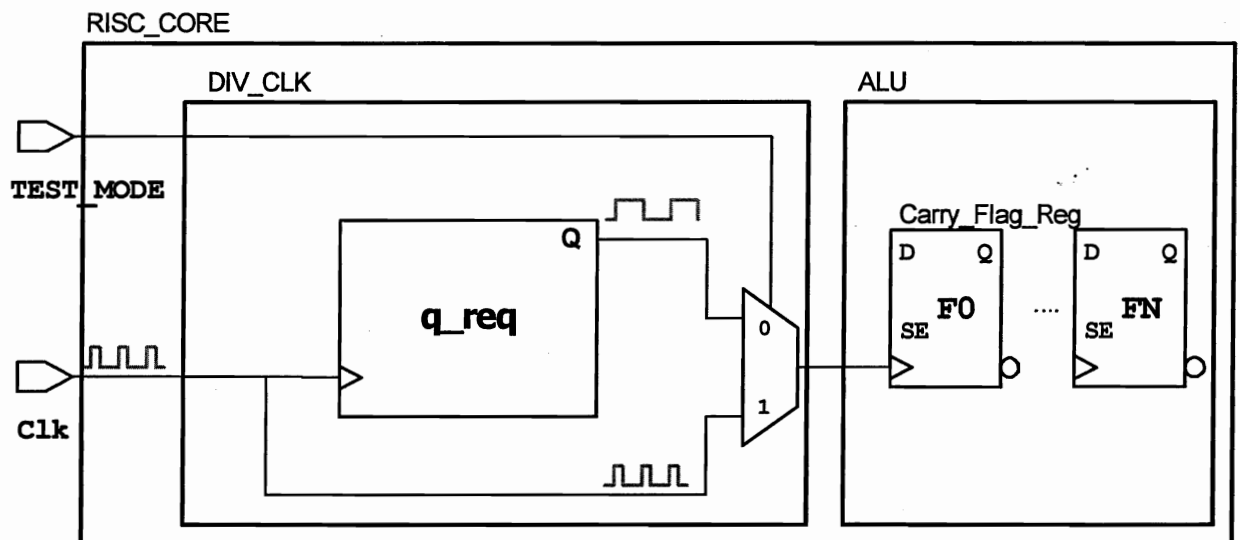
4 DFF set/reset line not controlled violations (D2)

85 DFF set/reset line not controlled violations (D3)

Question 2. What is the netlist name for the flip-flop on the left, the cause for the D1 violation?

I_DIV_CLK/q_reg

Question 3. Draw a schematic “fix” for this uncontrollable clock problem.



Note: The image above only shows I_DIV_CLK/q_req/Q going to the ALU block in RISC_CORE, but it is actually connected to several other blocks (a total of 310 FFs).

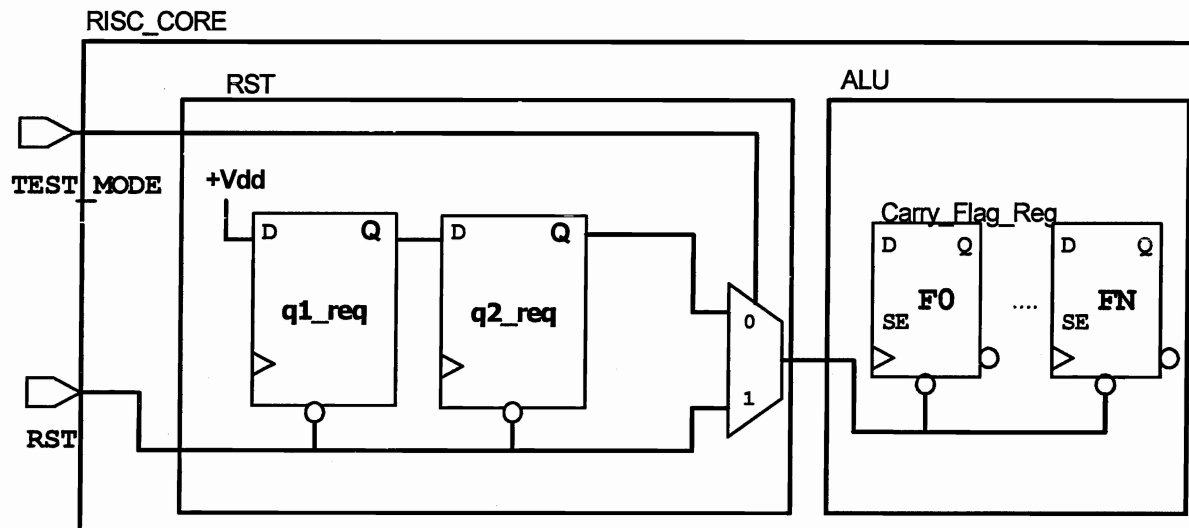
Question 4. What can you conclude about the D1 violations in the RISC_CORE design?

All the D1 violations are coming from the same launch point and have the same solution.

Lab 4

Answers / Solutions

Question 5. Draw a schematic “fix” for this uncontrollable reset problem.



Question 6. What can you conclude about the D3 violations in the RISC_CORE design?

The D3 violations all come from the same launch point and the solution in Question 5 is adequate to fix all of the D3 violations.

Task 3. AutoFix The D1 and D3 violations

Question 7. What kind of problems can AutoFix solve?

Uncontrollable clocks and sets/resets as well as improving coverage on the shadow logic around memories (shadow wrappers are covered in Unit 7).

3. Set up to run Autofix by modifying the 5preview_dft.tcl file.

Contents of file **autofix.tcl** (also available in the .solutions directory):

```
set_dft_configuration -fix_set enable \  
-fix_reset enable -fix_clock enable  
  
set_autofix_configuration -type reset -method mux  
  
set_dft_signal -view spec -type TestMode \  
-port TEST_MODE
```

```
# use the clock Clk as the autofix clock
set_dft_signal -view spec -port Clk -type TestData
set_autofix_configuration -type clock \
                        -control TEST_MODE -test_data Clk

# use the Reset line as the reset fix
set_dft_signal -view spec -port Reset -type TestData
set_autofix_configuration -type reset \
                        -control TEST_MODE -test_data Reset
```

```
***** Test Point Plan Report *****
Total number of test points : 3
Number of Autofix test points: 3
Number of Wrapper test points: 0
Number of test modes : 1
Number of test point enables : 0
Number of data sources : 2
Number of data sinks : 0
*****
```

Dft signals:

```
TestMode: TEST_MODE (no hookup pin)
TestData: Clk (no hookup pin)
TestData: Reset (no hookup pin)
```

Question 8. How many AutoFix test points are there? Why?

3 AutoFix test points. Because of the uncontrollable clock in the DIV_CLK block and the uncontrollable reset coming from the RST block. Two separate test points are used to fix the resets which makes the total 3.

Question 9. What was the number of violations?
Were there any cells with violations?
What was the fault coverage?

0 violations. No cells with violations. 99.73%

Question 10. What kind of fault coverage would you expect to get, if you had not autofixed the design?

Very low (0.36%) because almost every flip-flop in the design will not be included on the scan chain.

This page was intentionally left blank.

6

Top-Down Scan Insertion

Learning Objectives

After completing this lab, you should be able to:

- Use the `preview_dft` command to evaluate different scan architectures and iterate by revising scan specifications until you produce balanced top-level scan chains.
- Use the `preview_dft` command to confirm your scan specification for desired scan signals was applied correctly.



Lab Duration:
45 minutes

Lab 6

Overview

During this lab you will:

1. Preview the results of different scan specifications before implementing one that yields well-balanced top-level chains.
2. Perform the Mapped DFT flow for a test-ready design through scan insertion and test coverage estimation.

Unmapped Flow:

1. Read the RTL design into DFT.
2. Create and save a test protocol; verify that the design and the test protocol are compatible.
3. Compile and save the gate-level design; exit.

Mapped Flow:

4. Read the gate-level design and test protocol into DFTC.
5. Specify scan constraints and preview the scan architecture that will result from applying them.
6. Insert the scan chain.
7. Hand off the design and related files for use by downstream tools; exit.

File Locations

All files for this lab except for designs and libraries are located in the directory **lab6_topdown**.

Directory Structure

lab6_topdown	Current working directory
analyzed	Intermediate acs_read_hdl files
logs	Session log files
unmapped	Unmapped protocol
mapped	Gate-level netlist
mapped_scan	Gate-level scan design netlist
reports	DFTC reports
tmax	Files for downstream tools
scripts	Constraint and run scripts
../ref/rtl/ORCA/vhdl	Design files
../ref/db	Library files

Lab 6

Instructions

Your goal is to complete and run the first three steps of the Mapped Flow portion of the DFT Insertion Flow. The three you will modify are for reading the test-ready design and test protocol, previewing the scan architecture, inserting scan chains and finally, estimating test coverage.

The three scripts:

```
Unmapped Flow
  1read_design.tcl
  2create_test_protocol.tcl
  3compile_and_save.tcl
Mapped Flow
  4read_gates_and_protocol.tcl
  5preview_dft.tcl
  6insert_dft.tcl
  7handoff.tcl
```

Task 1. Read gate-level Design and its Test Protocol

1. Invoke DFTC.

```
cd lab6_topdown
dc_shell-t | tee logs/lab6.log
```

2. Create a shortcut to make running the scripts easier.

```
alias s "source -echo -verbose"
```

3. Examine and execute the script that performs step 4 of the DFT flow: reading in the gate-level design and the saved protocol.

```
exec cat scripts/4read_gate_and_protocol.tcl

s scripts/4read_gate_and_protocol.tcl
```

4. Check the scan state of the current design using this command:

```
report_scan_state
```

Question 1. If the Unmapped Flow ended with a compile –scan command, what should the scan state of the design you just read in be? Is that the design’s current scan state?

.....

At the end of the `4read_gate_and_protocol.tcl`, a DFT check was performed.

5. Examine the last several lines in your transcript to review the DFT check results for the test-ready design.

Question 2. What percentage of the total number of flip-flops are valid scan cells?

.....

Question 3. Considering just that percentage and no other factors, what range of estimated fault coverage would you predict for this design?

.....

Note: Step 4 of the DFT flow read the gate-level design and the test protocol. The `dft_drc` command used the saved test protocol for DFT design rule checking. The test protocol is one set of information and the scan specifications that may have been used to originally create the test protocol is another set of information.

6. Check this by reporting which dft signals are currently defined.

```
report_dft_signal
```

Question 4. How many ScanClocks and Resets are reported?

.....

Task 2. Specify and Preview Scan Architectures

1. Display the default scan architecture that will be implemented for this design.

Lab 6

```
preview_dft
```

- Question 5.** How many scan chains will be inserted?
What are the names of the preview scan chains?
Will the chains be well-balanced?

.....
.....
.....

- Question 6.** Will scan insertion use existing functional pins as the scan signals (scan in, scan out for each scan chain) or will it create new ones (test_si, test_so ports)?

.....

2. Display more scan architecture detail about the test clocks used for each scan chain.

```
preview_dft -show scan_clocks
```

Note: The ORCA design has only 3 clocks in test mode: pclk, sys_clk and sdr_clk.

- Question 7.** Why are four scan chains previewed by default?

.....

3. Use another `preview_dft` option and compare the content of the former and current `preview_dft` output.

```
preview_dft -show scan_summary
```

Question 8. Is there any information in the `scan_clocks` preview that is not also reported in the `scan_summary` preview?
When might you want to use the format of the `scan_summary` preview rather than the first?

.....
.....
.....

4. Try mixing the rising and falling `sdr_clk` scan flip-flops into the same chain.

```
set_scan_configuration -clock_mixing mix_edges
preview_dft -show scan_clocks
```

Question 9. How many scan chains are previewed now and are they now balanced?

.....

5. Allow mixing of different clock domain scan flip-flops into the same scan chain.

```
set_scan_configuration -clock_mixing mix_clocks
preview_dft -show scan_clocks
```

Question 10. How many scan chains are previewed now and what do you think the default `set_scan_configuration` `chain_count` value is?

.....

Question 11. What specifications would yield 6 balanced scan chains?

.....

Lab 6

6. Apply those specifications (6 chains) and confirm using `preview_dft`.

```
set_scan_configuration    # specify your options
preview_dft -show scan_clocks
```

Question 12. What is the difference in scan chain length between the longest and shortest chains now?

.....
.....

Question 13. Will DFTC insert any lock-up latches into this scan design? Which chains will include them?

.....
.....

Note: The functional pins of ORCA to use for scan are specified in the `settings_insert_dft.tcl` script.

7. Source that script and use `preview_dft` to answer the following questions.

```
s scripts/settings_insert_dft.tcl
preview_dft
```

Question 14. What are the names of the previewed scan chains now and what command defined their new names? Why were the chains given explicit names?

.....
.....

Question 15. Which pins are used for scan in, scan out and scan enable?

.....
.....

Task 3. Insert Scan Chains and Estimate Test Coverage

One of the commands in the settings `_insert_dft.tcl` script avoids all the subdesigns being renamed during the scan insertion process.

Question 16. What is the name of that command?

.....

Note: Use the man pages if needed to find the option to this command that disables all the gate-level optimization that occur by default during the last phases of the `_insert_dft` process.

Question 17. Which option to this command will disable gate-level optimization?

.....

1. Apply those two options whose effects will be to speed-up the runtime of the subsequent `_insert_dft` and to limit the number of changes to the existing design.

```
set_dft_insertion_configuration . . .
```

2. Perform scan insertion using the script provided.

```
s scripts/6insert_dft.tcl
```

Question 18. What is the estimated test coverage for the ORCA scan design?

.....

Note: Actual ATPG runs in TetraMAX obtained about 99% test coverage for this design. Doing so though required using functional RAM models and both TetraMAX's Fast Sequential and Full Sequential engines. Using DFT techniques to avoid the S30 violation, for example, allows that very high coverage much easier to obtain by merely using the Fast Sequential engine.

Question 19. How does this estimate compare to your earlier prediction?

.....

Lab 6

Note: When obtaining the estimated fault coverage for a scan inserted design, `dft_drc` performs Post-DFT checking and reports the DFT violations using the same ones that TetraMAX does.

Note: You can obtain detailed information for both the D* PRE-DFT violations and the {C,S,Z,X}* Post-DFT violations using the `man` command in DFTC.

3. Scroll back from the coverage report and answer the following questions.

Question 20. The S category of violations cannot be reported during Pre-DFT checks, why?

.....

Question 21. What does the S22 violation mean and does this explain why 2 Sequential Cells are reported as synchronization elements in the **Sequential Cells Without Violations** summary?

.....

.....

Congratulations! You have completed the lab.

Answers / Solutions

Task 1. Read gate-level Design and its Test Protocol

Question 1. If the Unmapped Flow ended with a `compile -scan` command, what should be the scan state of the design you just read in? Is that the design's current scan state?

The scan state should be "test-ready" which is reported as "scan cells replaced with loops".

Question 2. What percentage of the total number of flip-flops are valid scan cells?

$2925 / (2925 + 33)$ equals 98.9%

Question 3. Considering just that percentage and no other factors, what range of estimated fault coverage would you predict for this design?

Close to 99% test coverage.

Question 4. How many ScanClocks and Resets are reported?

Three ScanClocks (`sys_clk`, `sdr_clk`, and `pclk`) and one Reset (`prst_n`) are reported. The DFT signals are defined when the test protocol is read with the `read_test_protocol` command.

Task 2. Specify and Preview Scan Architectures

Question 5. How many scan chains will be inserted?
What are the names of the preview scan chains?
Will the chains be well-balanced?

4 scan chains

Named '1', '2', '3', and '4'.

No, the longest scan chain has 1074 cells while the shortest chain has half that many, 512, a very unbalanced scan chain architecture.

- Question 6.** Will scan insertion use existing functional pins as the scan signals (scan in, scan out for each scan chain) or will it create new ones (test_si, test_so ports)?
- New test_si and test_so ports are previewed because you have not yet specified your scan signals.
- Question 7.** Why are four scan chains previewed by default?
- Default is to not mix clocks or edges on scan chains. There are three clock domains. The sdr_clk domain has both rising-edge and falling-edge triggered flip-flops.
- Question 8.** Is there any information in the scan_clocks preview that is not also reported in the scan_summary preview? When might you want to use the format of the scan_summary preview rather than the first?
- The previewed information is almost the same. The key difference is that the scan_clocks preview gives the head and tail of each scan chain, plus the first scan flip-flop whenever the scan path crosses clock domains. The scan_summary only previews the head scan flip-flop. The scan_summary format of the preview report might be easier to parse with Perl or TCL.
- Question 9.** How many scan chains are previewed now and are they now balanced?
- Three chains, still unbalanced with the longest chain having 1132 cells and the shortest chain having 719 cells.
- Question 10.** How many scan chains are previewed now and what do you think the default set_scan_configuration \ -chain_count value is?
- Only one chain, the default for -chain_count.
- Question 11.** What specifications would yield 6 balanced scan chains?
- ```
set_scan_configuration \
-chain_count 6 -clock_mixing mix_clocks
```
- Question 12.** What is the difference in scan chain length between the longest and shortest chains now?
- Only one, with the longest chains being 488 cells long and the shortest 487.

**Question 13.** Will DFTC insert any lock-up latches into this scan design? Which chains will include them?

Yes.

Chain '4' has both pclk and sdr\_clk rising-edge flops at time 45 and Chain '5' has both sdr\_clk and sys\_clk rising-edge flops at time 45.

**Question 14.** What are the names of the previewed scan chains now and what command defined their new names? Why were the chains given explicit names?

Chain names: chain0, chain1, chain2, chain3, chain4 and chain5.

The set\_scan\_path command was used to define the scan chain names.

To associate a given pair of ScanDataIn and ScanDataOut scan signals to a specific chain. You need to specify the name of chain when you apply the set\_scan\_path command.

**Question 15.** Which pins are used for scan in, scan out and scan enable?

pads[0] to pad[5] are the scan in pins and sd\_A[0] to sd\_A[5] are the corresponding scan out pins. The scan enable port is called scan\_en.

### Task 3. Insert Scan Chains and Estimate Test Coverage

**Question 16.** What is the name of that command?

set\_dft\_insertion\_configuration

**Question 17.** Which option to this command will disable gate-level optimization?

```
set_dft_insertion_configuration -synthesis none \
 -preserve_design_name true
```

**Question 18.** What is the estimated test coverage for the ORCA scan design?

95.56%

- Question 19.** How does this estimate compare to your earlier prediction?
- This is probably lower than what you predicted, likely 99%
- Question 20.** The S category of violations cannot be reported during Pre-DFT checks, why?
- There are no scan chains then, and the S rules are related to problems with scan chain shifting.
- Question 21.** What does the S22 violation mean and does this explain why 2 Sequential Cells are reported as synchronization elements in the **Sequential Cells Without Violations** summary?
- S22 warns that ORCA has scan chains that are driven by more than one test clock. Scan chains chain3 and chain4 have lock-up latches and thus avoid the S22 violation. The S22 warnings about a negative-edge triggered scan flop in the sdr\_clk domain being stitched directly to a rising-edge triggered scan flop in the pclk domain.

# 7

## Scan Design Handoff and Custom Test Initialization Protocols

### Learning Objectives

After completing this lab, you should be able to:

- Name the 3 types of DFTC output files downstream tools require specifically related to scan testing.
- Explain what types of designs require custom test protocols.
- Write out the final test protocol and a scandef file.



**Lab Duration:**  
**75 minutes**

## Lab 7

# Overview

### Unmapped Flow:

1. Read the RTL design into DFT.
2. Create and save a test protocol; verify that the design and the test protocol are compatible.
3. Compile and save the gate-level design; exit.

### Mapped Flow:

4. Read the gate-level design and test protocol into DFTC.
5. Specify scan constraints and preview the scan architecture that will result from applying them.
6. Insert the scan chain.
7. Hand off the design and related files for use by downstream tools; exit.

### Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, check the back portion of this lab for help.

## File Locations

All files for this lab except for designs and libraries are located in the directory **lab7\_export**.

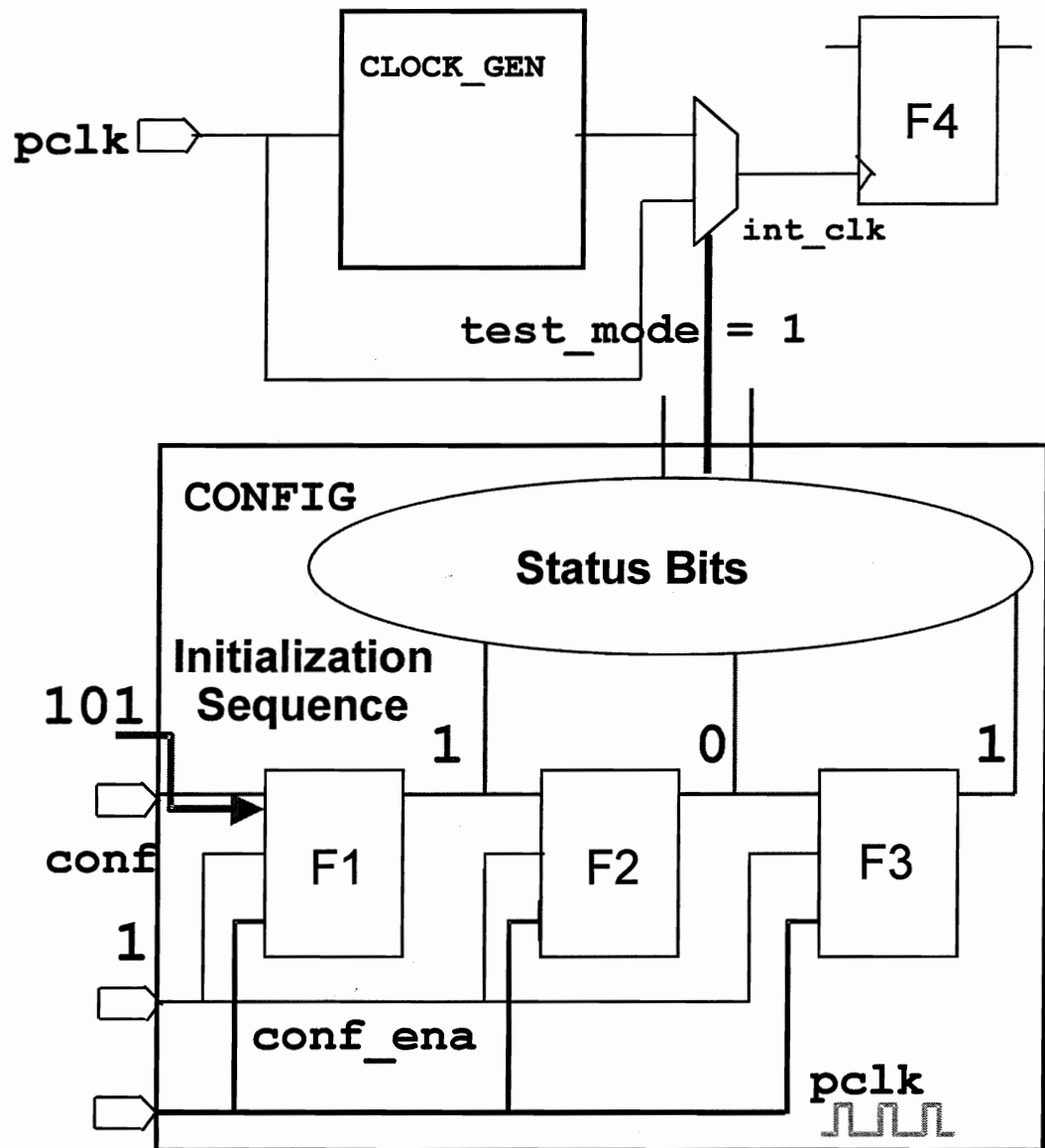
### Directory Structure

|                       |                                 |
|-----------------------|---------------------------------|
| <b>lab7_export</b>    | Current working directory       |
| analyzed              | Intermediate acs_read_hdl files |
| logs                  | Session log files               |
| unmapped              | Unmapped protocol               |
| mapped                | Gate-level netlist              |
| mapped_scan           | Gate-level scan design netlist  |
| reports               | DFTC reports                    |
| tmax                  | Files for downstream tools      |
| scripts               | Constraint and run scripts      |
| ../rtl/ORCA_init/vhdl | Design files                    |
| ../libs               | Library files                   |

## Lab 7

### Design Introduction

The design in this lab requires a custom test initialization protocol. You obtained the following schematic from the ORCA design documentation that describes the process for asserting the internal test\_mode signal.





## Lab 7

Since DFTC cannot infer a test protocol of this type you must develop one for this design. Answer the following questions in preparation for doing the subsequent lab tasks.

**Question 1.** What value must be placed on the `conf_ena` signal to initialize the configuration register flip-flops F1, F2 and F3?

.....

**Question 2.** How many cycles must you pulse `pclk` before `test_mode` is asserted?

.....

**Question 3.** What values should `conf_ena` and `pclk` have the cycle after `test_mode` is asserted?

.....

**Question 4.** Write in STIL format the Vector statements that would go into the “`test_setup`”?

```
. . .
MacroDefs {
 "test_setup"
 {
 . . .
```

```
. . .
}
```

You will use this information later when you develop the initialization protocol for ORCA.

## Lab 7

# Instructions

During this lab you will, develop a custom test initialization protocol for a design that has an internal `test_mode` signal. Given the gate-level scan version of that design export from DFTC the files required by downstream tools such as ATPG and P&R.

Your goal is to modify three of the seven scripts that perform the seven basic steps in running DFTC. The three you will modify focus on the development of a custom test protocol required for the ORCA design, and the handoff of all the necessary scan design files.

The seven scripts:

Unmapped Flow:

```
1read_design.tcl
2create_test_protocol.tcl
3compile_and_save.tcl
```

Mapped Flow:

```
4read_gates_and_protocol.tcl
5preview_dft.tcl
6insert_dft.tcl
7handoff.tcl
```

## Task 1. Execute the Unmapped Flow

---

1. Invoke DFTC.

```
cd lab7_export
dc_shell-t
```

2. Create a shortcut to make running the scripts easier.

```
alias s "source -echo -verbose"
```

3. Step 1 of the DFT flow has already been done for you in the interest of saving time during lab (Reading the RTL).

Examine the contents of the script that reads the saved ddc file for the ORCA RTL design and then source the script to read in ORCA.

```
exec cat scripts/1read_design.tcl

s scripts/1read_design.tcl
```

- Question 5.** Where have the results of the reading RTL process been saved?
- .....

*Before you create the starting test protocol you must specify all your clocks, resets, constants, etc. The clocks and the resets are already defined in the **settings\_protocol.tcl** script.*

- Question 6.** After the test initialization asserts the internal `test_mode` signal what must you do to keep the ORCA design in that mode throughout the rest of the scan testing?
- .....

4. Add the appropriate command to the **settings\_protocol.tcl** script.
5. Create the test protocol for ORCA by running the script provided.

```
s scripts/2create_test_protocol.tcl
```

Review the results of the `dft_drc` run on the ORCA RTL design.

## Lab 7

**Question 7.** Are there few or many testability problems reported?

.....

*Developing the initialization test protocol from scratch is not practical for most engineers. A better method is to modify the protocol files that DFTC or TetraMAX create for you.*

6. Save out the current test protocol for the design.

```
write_test_protocol -o unmapped_flow.spf
vi unmapped_flow.spf
```

7. Add the required Vector statements to the unmapped\_flow.spf.

**Question 8.** Since you only changed the “test\_setup” section of the STIL test protocol, which command should you use to read it back into DFTC?

.....

8. Verify that your initialization vectors are correct by reading in your custom protocol and rerunning dft\_drc.

**Question 9.** What happens when you first try to read unmapped\_flow.spf back into DFTC?

.....

9. Solve the problem by removing the previous protocol and reading in your custom initialization.

```
remove_test_protocol
read_test_protocol -section test_setup \
 unmapped_flow.spf
```

**Note:** The “test\_setup” or initialization section is only part of a complete test protocol.

## Lab 7

10. Re-create the rest of the test protocol before attempting to run `dft_drc`.

```
create_test_protocol
dft_drc
```

**Question 10.** How do the results of `dft_drc` compare to your first run?

.....

11. Now that you have validated that the custom initialization protocol is working, write out the complete protocol for later use in the Mapped Flow.

```
write_test_protocol -out unmapped_flow.spf
```

*Step 3 of the DFT flow (compile) has already been done for you in the interest of saving time.*

12. Examine the contents of the script that compiles the RTL for the ORCA design into a test-ready design.

```
exec cat scripts/3compile_and_save.tcl
```

**Question 11.** Where have the results of the test-ready compile been saved?

.....

## Lab 7

### Task 2. Execute the Mapped Flow

---

1. Read the gate-level design and test protocol into DFTC.

```
s scripts/4read_gate_and_protocol.tcl
```

**Question 12.** What, if anything, is different from the `dft_drc` results on the test-ready gates versus the previous `dft_drc` on the RTL?

.....

2. Specify scan constraints and preview the scan architecture that will result from applying them by using the script provided.

```
s scripts/settings_insert_dft.tcl
preview_dft
```

**Question 13.** Explain why the 3 configuration registers will not be placed in the scan chains?

.....

.....

3. Insert the scan chain.

```
s scripts/6insert_dft.tcl
```

**Question 14.** What is the estimated test coverage for the scan design?

.....

## Lab 7

4. Complete the script that writes out the design and related files to be used by downstream tools (scripts/7handoff.tcl).

Add to this script:

- the command to save the needed ATPG spf file (to the ./tmax directory) that is not currently being saved.
- the command to write out the scan chains to a SCANDEF file (to the mapped\_scan directory).

```
unix% vi scripts/7handoff.tcl
```

5. Execute the handoff script and examine that additional file.

```
s scripts/7handoff.tcl
```

**Question 15.** Is the custom initialization sequence still there?  
What has changed about it?

.....  
.....

The test protocol for preview\_dft contains your scan chain in and out signal declarations.

**Question 16.** What additional information has DFTC added to the ScanStructures section now that the protocol is for a scan inserted design?

.....

**Question 17.** How many scanchains does the scandef file show, and how many partitions were created (use grep)?

.....

**Congratulations!** You have completed the lab.

## Answers / Solutions

Solution scripts are available in the `.solutions` directory.

**Question 1.** What value must be placed on the `conf_ena` signal to initialize the configuration register flip-flops F1, F2 and F3?

`conf_ena` must be logic 1 to initialize the flip-flops.

**Question 2.** How many cycles must you pulse `pclk` before `test_mode` is asserted?

`pclk` must pulse three times.

**Question 3.** What values should `conf_ena` and `pclk` have the cycle after `test_mode` is asserted?

Both should be logic 0.

**Question 4.** Write in STIL format the Vector statements that would go into the “test\_setup”?

```
. . .
MacroDefs {
 "test_setup"
 {
 . . .
 V { "conf_ena"=1; "conf"=1; "pclk"=P; }
 V { "conf_ena"=1; "conf"=0; "pclk"=P; }
 V { "conf_ena"=1; "conf"=1; "pclk"=P; }
 V { "conf_ena"=0; "pclk"=0; }
 . . .
 }
}
```

**Question 5.** Where have the results of the reading RTL process been saved?

`unmapped/ORCA.ddc`

**Question 6.** After the test initialization asserts the internal `test_mode` signal what must you do to keep the ORCA design in that mode throughout the rest of the scan testing?

You must hold the `conf_ena` at logic 0 by using the following command:

```
set_dft_signal -view existing_dft -type Constant \
 -active_state 0 -port conf_ena
```



**Question 7.** Are there few or many testability problems reported?

Many.

## 4010 PRE-DFT VIOLATIONS

- 2828 Uncontrollable clock input of flip-flop violations (D1)
- 1133 DFF set/reset line not controlled violations (D3)
- 32 Clock feeding both clock and data input violations (D11)
- 6 Clock feeding multiple clock/set/reset inputs violations (D12)
- 1 Clock path affected by clock captured by clock in trailing edge clock\_port violation (D16)
- 10 Clock\_port not capable of capturing data violations (D17)

7. Add the required vector statements to the unmapped\_flow.spf.

```
V { "conf"=1; "conf_ena"=1; "pclk"=P; }
V { "conf"=0; "conf_ena"=1; "pclk"=P; }
V { "conf"=1; "conf_ena"=1; "pclk"=P; }
V { "conf_ena"=0; "pclk"=0; }
```

**Question 8.** Since you only changed the “test\_setup” section of the STIL test protocol, which command should you use to read it back into DFTC?

```
read_test_protocol -section test_setup
```

**Question 9.** What happens when try to read unmapped\_flow.spf back into DFTC?

```
dc_shell-xg-t> read_test_protocol -section test_setup unmapped_flow.spf
Warning: a protocol already exists in mode 'all_dft.' Please
use remove_test_protocol command to delete the existing test protocol.
(TEST-1402)
```

**Question 10.** How do the results of dft\_drc compare to your first run?

Fewer PRE-DFT violations, but new “cell is constant” violations that reflect the 1-0-1 initialization sequence.

## 94 PRE-DFT VIOLATIONS

- 32 Clock feeding both clock and data input violations (D11)
- 6 Clock feeding multiple clock/set/reset inputs violations (D12)
- 36 Data path affected by clock captured by clock in trailing edge clock\_port violations (D14)
- 20 Clock\_port not capable of capturing data violations (D17)

## 3 OTHER VIOLATIONS

- 1 Cell is constant 0 violation (TEST-504)
- 2 Cell is constant 1 violations (TEST-505)

**Question 11.** Where have the results of the test-ready compile been saved?

mapped/ORCA.ddc

**Question 12.** What, if anything, is different from the `dft_drc` results on the test-ready gates versus the previous `dft_drc` on the RTL?

When running at the gate-level `dft_drc` also reports a summary Sequential Cell Report.

```
Sequential Cell Report
36 out of 2961 sequential cells have violations
...
```

**Question 13.** Explain why the 3 configuration registers will not be placed in the scan chains?

The PRE-DFT violations of “cell is constant 0/1” mean DFTC cannot shift arbitrary logic 0s and 1s through these cells as required for scan shift; they are always 0 or always 1.

**Question 14.** What is the estimated test coverage for the scan design?

The test coverage is around 92%.

**Question 15.** Is the custom initialization sequence still there? What has changed about it?

Yes. In the `unmapped_flow.spf` file, the `test_setup` vectors you added may have specified the same values for the pins for adjacent Vector statements. In the STIL file DFTC wrote for TetraMAX, notice “`pclk`”=P is only done once, for the first configuration cycle.

**Question 16.** What additional information has DFTC added to the ScanStructures section now that the protocol is for a scan inserted design?

The ScanStructures section now includes for each chain a `ScanLength` specification that declares how many scan cells are in each chain.

**Question 17.** How many scanchains does the `scandef` file show, and how many partitions were created?

Use “`grep PARTITION mapped_scan/ORCA.scandef`”.

9 scan chains and 4 partitions.

# 9

## Improving Scan Insertion Run-Time and Capacity

### Learning Objectives

After completing this lab, you should be able to:

- Given existing scan insertion scripts, implement RSS in a top-down scan insertion flow achieving well-balanced scan chains
- Modify a bottom-up scan insertion script for full gate-level designs to use test models/ILMs with RSS and run it
- Preview top-level chain balance using test models/ILMs after block level scan insertion and revise the block level scan architecture as needed to improve top-level scan chain balance

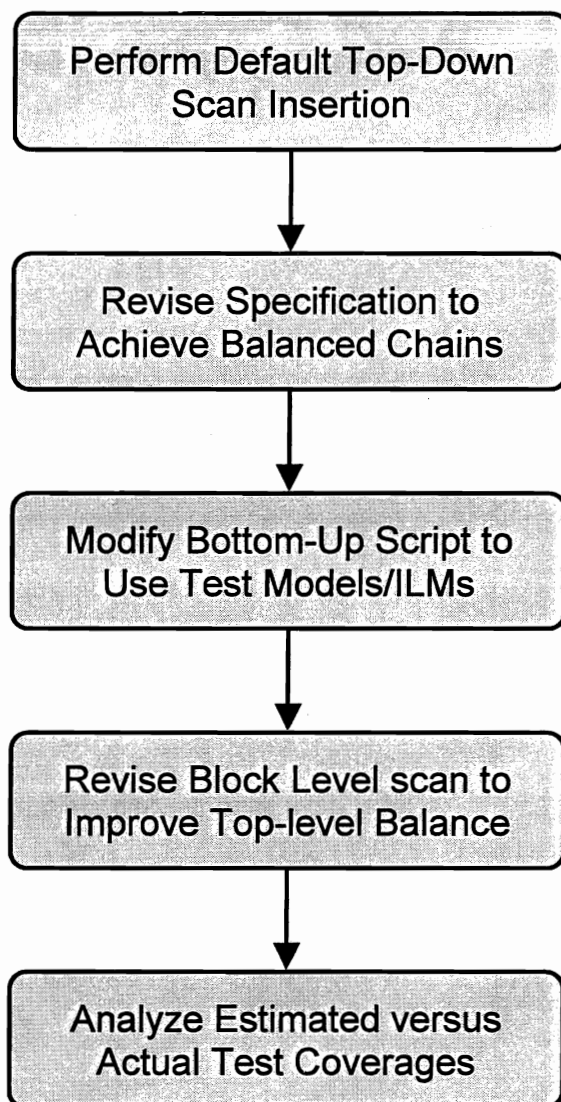


**Lab Duration:**  
75 minutes

## Lab 9

### Overview

#### Lab 9 Tasks



#### Answers & Solutions

This lab guide contains answers and solutions to all questions. If you need some help with answering a question, check the back portion of this lab for help.

# File Locations

All files for this lab are located in the directory **lab9\_hicap**.

## Directory Structure

|                           |                                     |
|---------------------------|-------------------------------------|
| <b>lab9_hicap</b>         | Current working directory           |
| dc_ilms                   | ILMs with test models for subblocks |
| mapped                    | ORCA design and its subblocks       |
| mapped_scan               | Scan-inserted design and subblocks  |
| reports                   | DFT reports                         |
| scripts                   | Scan insertion scripts              |
| test_models               | Test models for subblocks           |
| tmax                      | TetraMAX data                       |
| <b>.solutions/scripts</b> | Solutions for this lab              |

## Relevant Files

|                                  |                                  |
|----------------------------------|----------------------------------|
| <b>scripts/</b>                  |                                  |
| bottom_up.tcl                    | Bottom-up scan insertion script. |
| Calls the following two scripts: |                                  |
| scan_blocks.tcl                  | Inserts scan in to the subblocks |
| scan_top.tcl                     | Inserts scan in to the top-level |
| top_down.tcl                     | Top-Down scan insertion script   |

## Lab 9

# Instructions

During this lab you will perform a top-down and a bottom-up scan insertion flow on the design ORCA. Some scripts are already provided. You will modify the scripts for the various tasks.

These scripts contain almost all the DFTC commands you would need to use when working on a real-world design back in your office. After completing this lab, you will have practiced the steps needed to improve both the run-time and the capacity of your scan insertion scripts. When you re-use the scripts back at your office focus on the design-specific items (names of clocks, resets, and so forth).

The lab starts with a top-down script since it is easier to understand and provides reference data (number and length of top-level scan chains, estimated test coverage) that you can later use to compare the results of the bottom-up scan insertion script.

### Task 1. Top-Down Scan Insertion using RSS

---

1. Make sure your current directory is **lab9\_hicap**:

```
UNIX% cd lab9_hicap
```

2. Open the file `scripts/top_down.tcl` in your favorite editor (`vi`, `emacs`, etc.).

The `top_down.tcl` script is used to insert scan in a top-down fashion on the ORCA design.

```
UNIX% vi scripts/top_down.tcl
```

3. Modify this script to use the RSS features you learned about in lecture.

Compared to the original script in the lab directory, your revised script should:

- a. Modify the original design as little as possible
- b. Run scan insertion faster
- c. Consume less memory

If needed, check the Answers section for hints.

Once you are done, save the modified script file.

4. Start Design Compiler from within the **lab9\_hicap** directory:

```
UNIX% dc_shell-t | tee logs/lab9.log
```

5. Execute the script you just edited:

```
s scripts/top_down.tcl
```

6. Open the report generated by `preview_dft` and analyze it.

**Question 1.** Are the scan chains well balanced? If not, why?

.....

**Note:** It is important to balance the scan chains if tester time and the cost of test are to be reduced.

7. Open the **top\_down.tcl** file again and **change** it so that you can achieve balanced top-level scan chains.
8. Remove the first pass of scan design files before improving the scan architecture.

```
remove_design -design
```

9. Rerun the `top_down.tcl` script in DFTC.

**Question 2.** Are the scan chains more balanced now?

.....

**Question 3.** What DFT structure did DFTC have to add to achieve these balanced scan chains?

.....

**Note:** The ORCA design has only 3 test clocks, but your real-world design might have many more test clocks. The more test clocks, the greater the probability for having more of these DFT structures. If minimizing die area is an important design requirement, you may need to monitor their number.

## Lab 9

10. Determine the number of lockup latches DFTC inserted. DFTC names the latches using an intuitive naming convention that is good for wildcard searches.

```
get_cells LOCKUP* -hier
```

**Question 4.** How many lockup latches were added?

.....

**Note:** If your design has many lockup latches counting them yourself can become tedious. Have DFTC count them using:

```
sizeof_collection [get_cells LOCKUP* -hier]
```

**Question 5.** Where in the ORCA hierarchy were they placed?

.....

**Note:** If the ORCA design will be placed and routed using a top-down methodology, the placement of the lockup latches is probably not important. If the ORCA design will be placed and routed in a bottom-up fashion, however, you may have preferred locations for them. For example, you may want to avoid any lockup latches at the top-level.

11. Obtain an estimate of the design's test coverage in DFTC.

**Question 6.** What is the estimated test coverage for ORCA?

.....

12. Exit Design Compiler.



## Task 2. Bottom-Up Scan Insertion using Test Models

Two scripts are used for bottom-up scan insertion, `scan_blocks.tcl` and `scan_top.tcl`. These scripts currently run using full gate-level netlists.

In the subsequent steps you will first run **`scan_blocks.tcl`** to insert scan into the blocks of the ORCA design generating test models for the blocks. Later you will run the **`scan_top.tcl`** to complete the top-level scan chain stitching in ORCA using the test models created for the blocks instead of the full gate-level netlists.

The ORCA top-level design contains all the I/O pad instantiations, a `CLOCK_GEN` block that contains the PLLs and clock multipliers, and an `ORCA_TOP` block that serves as the core of the design and contains all the blocks for which you will perform scan insertion.

The `foreach` loop in the `scan_blocks.tcl` script iterates through a TCL list of the blocks within `ORCA_TOP`. The loop first reads in the appropriate `.ddc` file from the correct directory and then proceeds with block-level scan insertion. The results are saved at the end of the loop and all design files are removed before proceeding to the next block.

1. Locate the `foreach` loop within the `scan_blocks.tcl` script and modify it to save the test model for each block in a file separate from the DDC file in the `test_models` directory (put these commands in the script immediately before the command that removes each block).
2. Modify the `scan_top.tcl` to read in the test models only from the proper directory (these test models were saved by the `scan_blocks.tcl` script).

Read the test models first, and then read in the block that contains them, `ORCA_TOP`. Make sure the **`RESET_BLOCK`** is **not** a test model.

```

. . .
Your commands go here
. . .
change_names -rules verilog
read_ddc mapped/ORCA.ddc
current_design ORCA
link
. . .

```

## Lab 9

3. Examine the `scan_blocks.tcl` script and answer the following questions.

**Question 7.** Why specify `-clock_mixing no_mix` for block level scan insertion?

.....  
.....

**Question 8.** What is the result of `-insert_terminal_lockup` for block level scan insertion?

.....  
.....

**Question 9.** If lockup latches will be inserted at the end of all block level scan chains, do you need to insert any lockup latches between block level chains at the top-level?

.....

4. Modify the `scan_top.tcl` to disable lockup latch insertion, but allow `clock_mixing`.
5. Run the `bottom_up.tcl` script, which first runs `scan_blocks.tcl` on the blocks within `ORCA_TOP` and then runs `scan_top.tcl` on the ORCA design.

```
UNIX% dc_shell-t -f scripts/bottom_up.tcl | tee
bottom_up.log
```

6. Examine the top-level `preview_dft` report for ORCA.

```
vi reports/ORCA_preview_dft.rpt
```

**Question 10.** Are the scan chains balanced? If not, why?

.....

7. Use the approach outlined in lecture to revise the block level scan architectures so the top-level scan chains are better balanced (stop when all scan chain lengths are within 15 scan cells of each other).

**Question 11.** Which command and setting produced this better-balanced top-level scan architecture?

.....

**Question 12.** In the context of a design that contains test models, why can you NOT use the same method for counting the number of lockup latches in the design as was used in Task 1?

.....

.....

**Note:** Since the resulting Verilog gate-level netlists of both the block level scan insertion and the top-level scan stitching are saved to the same UNIX directory, you can use UNIX commands to count the lockup latches.

8. Determine the number of LOCKUP latches inserted throughout ORCA.

```
UNIX% fgrep LOCKUP tmax/*.v | wc -l
```

**Note:** fgrep searches for fixed string matches and wc -l outputs the number of matched lines. In this case, that number is equal to the number of lockup latches.

**Question 13.** How many lock-up latches were inserted?

.....

**Question 14.** Where in the ORCA hierarchy were they placed (Hint: do the fgrep without piping it to wc)?

.....

.....

.....

## Lab 9

**Note:**

If the ORCA design will be placed and routed using a top-down methodology, the placement of the lockup latches is probably not important. If the ORCA design will be placed and routed in a bottom-up fashion, however, you may have preferred locations for them. For example you may want to avoid any lockup latches at the top-level.

**Question 15.** How does this compare to the number of lock-up latches inserted in Task 1 by the top-down script?

.....

9. Examine the estimated coverage report for ORCA and answer the following questions.

```
UNIX% cat reports/ORCA_coverage.rpt
```

**Question 16.** How does this coverage compare to the estimated coverage obtained when inserting scan top-down into ORCA?

.....

.....

**Question 17.** Why is the estimated coverage obtained using test models not meaningful?

.....

.....

**Note:**

In order to obtain a more meaningful estimate for the test coverage, you must read the entire design into TetraMAX.

10. Run a script to obtain the actual test coverage in TetraMAX and then answer the following questions.

```
UNIX% cd tmax
UNIX% tmax -shell run.tmax
```

**Question 18.** Why is the total number of faults in TetraMAX much greater than the number of faults in DFTC?

.....

.....

**Question 19.** What test coverage does TetraMAX obtain for ORCA?

.....

**11.** Exit Design Compiler.

### **Task 3. Optional: Bottom-up Scan Insertion using ILMs with Test Models Attached**

---

- 1.** Repeat Task 2 but instead use ILMs with test models attached.
  - Modify scripts/ilm\_scan\_blocks.tcl to generate and save the ILMs.
  - Modify scripts/ilm\_scan\_top.tcl to read in the ILMs.

If you get stuck, check the .solutions directory.

**Congratulations!** This completes the lab.

## Answers / Solutions

### Task 1. Top-Down Scan Insertion using RSS

3. Modify this script to use the RSS features you learned about in lecture. Compared to the original script in the lab directory, your revised script should:
- Modify the original design as little as possible
  - Run scan insertion faster
  - Consume less memory

```
set_dft_insertion_configuration \
 -synthesis none \
 -preserve_design_name true
```

**Question 1.** Are the scan chains well balanced? If not, why?

No. The scan chains are very unbalanced because clock mixing is not enabled.

7. Open the top\_down.tcl file again and change it so that you can achieve balanced top-level scan chains.

```
set_scan_configuration -clock_mixing mix_clocks
```

**Question 2.** Are the scan chains more balanced now?

Yes. All 6 chains are either 488 or 487 elements long.

**Question 3.** What DFT structure did DFTC have to add to achieve these balanced scan chains?

Lockup latches.

**Question 4.** How many lockup latches were added?

2

**Question 5.** Where in the ORCA hierarchy were they placed?

```
dc_shell-xg-t> get_cells LOCKUP* -hier
{I_ORCA_TOP/I_RESET_BLOCK/LOCKUP I_ORCA_TOP/I_SDRAM_WRITE_FIFO/LOCKUP}
```

**Question 6.** What is the estimated test coverage for ORCA?

97.67%

## Task 2. Bottom-Up Scan Insertion using Test Models

1. Locate the foreach loop within the scan\_blocks.tcl script and modify it to save the test models only for each block:

```
foreach design $designs {
 . . .
 # Save the Test Model HERE
 write_test_model -format ddc \
 -output test_models/${design}.ddc
 . . .
}
```

2. Modify the scan\_top.tcl to read in the test models only from the proper directory (these test models were saved by the scan\_blocks.tcl script.)

Read the test models first, and then read in the block that contains them, ORCA\_TOP.

```
read_ddc [glob test_models/*.ddc]
use_test_model -true [get_designs *]
remove_design RESET_BLOCK
read_ddc mapped_scan/RESET_BLOCK.ddc
use_test_model -false RESET_BLOCK

read_ddc mapped/ORCA_TOP.ddc
change_names -rule verilog
read_ddc mapped/ORCA.ddc
current_design ORCA
link
. . .
```

**Question 7.** Why specify -clock\_mixing no\_mix for block level scan insertion?

Mixing clocks in block level scan chains would restrict top-level scan chain balancing. It is therefore better to postpone scan chain clock mixing until you perform the final top-level scan stitching.

**Question 8.** For block-level scan insertion, what is the result of using  
insert\_terminal\_lockup?

Each block level scan chain ends with a lockup latch, eliminating the need for lockup latches at the top-level.



**Question 9.** If lockup latches will be inserted at the end of all block-level scan chains, do you need to insert any lockup latches between block-level chains at the top-level?

No, doing so would create back-to-back lock-up latches in the scan chains.

5. Modify the scan\_top.tcl to disable lockup latch insertion, but allow clock\_mixing.

```
set_scan_configuration -clock_mixing mix_clocks
set_scan_configuration -add_lockup false
```

**Question 10.** Are the scan chains balanced? If not, why?

No. Some block level scan chains are much longer than others. This affects the balance of the top-level scan chains; they are very unbalanced.

**Question 11.** Which command and setting produced this better-balanced top-level scan architecture?

In scan\_blocks.tcl:

```
set_scan_configuration -max_length 200
```

**Question 12.** In the context of a design that contains test models, why can you NOT use the same method for counting the number of lockup latches in the design as was used in Task 1?

You cannot count the latches within a test model because the test model is an empty design in DFTC.

**Question 13.** How many lock-up latches were inserted?

27



**Question 14.** Where in the ORCA hierarchy were they placed  
(*Hint: do the fgrep without piping it to wc*)?

```
unix% fgrep LOCKUP tmax/*.v | awk '{printf ("%s %s %s\n", $1, $2, $3)}'
```

```
BLENDER_gates.v: lanlq1 LOCKUP1
BLENDER_gates.v: lanlq1 LOCKUP
CONTEXT_MEM_gates.v: lanlq1 LOCKUP
PARSER_gates.v: lanlq1 LOCKUP1
PARSER_gates.v: lanlq1 LOCKUP
PCI_CORE_gates.v: lanlq1 LOCKUP5
PCI_CORE_gates.v: lanlq1 LOCKUP4
PCI_CORE_gates.v: lanlq1 LOCKUP3
PCI_CORE_gates.v: lanlq1 LOCKUP2
PCI_CORE_gates.v: lanlq1 LOCKUP1
PCI_CORE_gates.v: lanlq1 LOCKUP
PCI_RFIFO_gates.v: lanlq1 LOCKUP1
PCI_RFIFO_gates.v: lanlq1 LOCKUP
PCI_WFIFO_gates.v: lanlq1 LOCKUP1
PCI_WFIFO_gates.v: lanlq1 LOCKUP
RISC_CORE_gates.v: lanlq1 LOCKUP1
RISC_CORE_gates.v: lanlq1 LOCKUP
SDRAM_IF_gates.v: lanhq1 LOCKUP5
SDRAM_IF_gates.v: lanlq1 LOCKUP4
SDRAM_IF_gates.v: lanhq1 LOCKUP3
SDRAM_IF_gates.v: lanlq1 LOCKUP2
SDRAM_IF_gates.v: lanhq1 LOCKUP1
SDRAM_IF_gates.v: lanlq1 LOCKUP
SDRAM_RFIFO_gates.v: lanlq1 LOCKUP1
SDRAM_RFIFO_gates.v: lanlq1 LOCKUP
SDRAM_WFIFO_gates.v: lanlq1 LOCKUP1
SDRAM_WFIFO_gates.v: lanlq1 LOCKUP
```

**Question 15.** How does this compare to the number of lock-up latches inserted in Task 1 by the top-down script?

There are many more lockup latches, one for each block level chain.

**Question 16.** How does this coverage compare to the estimate coverage obtained when inserting scan top-down into ORCA?

The 39.36% estimated test coverage is much lower than that obtained for the full gate-level top-down scan inserted design.

- Question 17.** Why is the estimated coverage obtained using test models not meaningful?
- A test model in DFTC is an empty design. When obtaining the estimated test coverage for a design in DFTC that uses test models, these empty designs will also be passed to TetraMAX.
- Question 18.** Why is the total number of faults in TetraMAX much greater than the number of faults in DFTC?
- TetraMAX has the full gate-level netlist for the entire design hierarchy. DFTC had the gate-level netlist for ORCA\_TOP and the designs above it (CLOCK\_GEN and ORCA); but all the blocks within ORCA\_TOP except for RESET\_BLOCK were test models (empty designs) in DFTC.
- Question 19.** What test coverage does TetraMAX obtain for ORCA?
- 98.36%

# 10

## DFT MAX flow

### Learning Objectives

After completing this lab, you should be able to:

- Insert Adaptive Scan using top down flow
- Understand needed components for Adaptive Scan
- Run TetraMAX flow on Adaptive Scan design



**Lab Duration:**  
**45 minutes**

## Lab 10

# Overview

During this lab you will:

1. Perform the Mapped DFT flow for a test-ready design through scan insertion while enabling Adaptive Scan insertion.
2. Identify needed files to export to TetraMAX.

Mapped Flow:

1. Read the gate-level design and test protocol into DFTC.
2. Specify scan constraints and preview the scan architecture that will result from applying them.
3. Add needed option to enable Adaptive Scan Flow.
4. Insert the scan chains and Adaptive Scan logic.
5. Understand post scan insertion dft\_drc checks.
6. Hand off the design and related files for use by downstream tools; exit.

## File Locations

All files for this lab except for designs and libraries are located in the directory **lab10\_max**.

### Directory Structure

|                      |                                |
|----------------------|--------------------------------|
| <b>lab10_max</b>     | Current working directory      |
| logs                 | Session log files              |
| mapped               | Gate-level netlist             |
| mapped_scan          | Gate-level scan design netlist |
| reports              | DFTC reports                   |
| tmax                 | Files for downstream tools     |
| scripts              | Constraint and run scripts     |
| ../ref/rtl/ORCA/vhdl | Design files                   |
| ../ref/db            | Library files                  |
| .solutions           | Solution scripts               |

## Lab 10

# Instructions

Your goal is to complete and run the four steps of the Mapped Flow portion of the DFT Insertion Flow. The scripts you will modify are for previewing the scan architecture, inserting scan chains and finally, handing off the needed files.

The four needed scripts:

```
Mapped Flow
4read_gates_and_protocol.tcl
5preview_dft.tcl
6insert_dft.tcl
7handoff.tcl
```

## **Task 1. Read gate-level Design and its Test Protocol**

1. Invoke DFTC.

```
cd lab10_max
dc_shell-xg-t | tee logs/mylab10.log
```

2. Examine and execute the script that performs step 4 of the DFT flow: reading in the gate-level design and creating the protocol.

```
exec cat scripts/4read_gate_and_protocol.tcl

s scripts/4read_gate_and_protocol.tcl
```

## Task 2. Specify and Preview Scan Architectures

1. Investigate side effects of specifying long chains.

```
set_scan_configuration \
 -clock_mixing mix_clocks -chain_count 2
preview_dft -show scan_summary
```

**Question 1.** How many scan chains will be inserted, and how will this affect the number of tester cycles on the tester?

.....

**Question 2.** What is one way to reduce Test Data Volume and Test Time?

.....

2. Change the **scripts/settings\_insert\_dft.tcl** script to enable compression with a compression ratio of 5.

```
set_dft_configuration -scan_compression enable
set_scan_compression_configuration -minimum_compression 5
set_scan_configuration -chain_count 5
```

**Note:** Since the design is so small for lab purposes, setting a compression ratio of 5 is enough to see the benefits.

**Question 3.** Do you need to specify the compression ratio?

.....

3. Do another `preview_dft` run and compare what was architected now.

```
preview_dft -show scan_summary
```

**Note:** For the class, a Tcl script is provided to display content in a separate window. Try “v preview\_dft -show scan\_summary”

**Question 4.** What do you see now in the preview report?

.....

## Lab 10

4. Investigate the use of the Test\_mode pin.

**Question 5.** Why was there a test\_mode pin added to the design?

.....

**Question 6.** Can you share the existing test\_mode pin that might be selecting Test Clocks?

.....

**Question 7.** Which pins are used for scan in, scan out and scan enable?

.....

5. Edit the file **scripts/settings\_insert\_dft.tcl** again and specify your own top level Test Mode pin for compression. Then rerun the script.

```
create_port -direction in TM_COMP
set_dft_signal -view spec -type TestMode -port TM_COMP
```

6. Since a new port was added you need to recreate the test protocol file and rerun dft\_drc.

```
remove_test_protocol
create_test_protocol
dft_drc
preview_dft -show scan_summary
```

**Question 8.** Which port is now being used to select between modes?

.....

7. Now insert\_dft to insert the chains and compression logic.

```
insert_dft
```

**Question 9.** What new messages do you see with insert\_dft?

.....



### Task 3. Hand-off files for ATPG for two modes

---

**Question 10.** How many STIL Test Protocol files do you need to write out with an Adaptive Scan flow?

.....

**Question 11.** How many netlists are needed?

.....

1. Check drc rule violations on Internal\_scan mode:

```
current_test_mode Internal_scan
dft_drc
```

2. Check drc rule violations for ScanCompression mode:

```
current_test_mode ScanCompression_mode
dft_drc
```

**Question 12.** What are the differences that you see?

.....

**Note:** Internal\_scan mode is also known as reconfigured scan mode.

3. Edit the file **scripts/7handoff.tcl** and add the command to write out the ScanCompression mode protocol.
4. Run the handoff script:

```
s scripts/7handoff.tcl
```

**Question 13.** Look at both test protocol files, do you see a difference?

.....

## Lab 10

### Task 4. Run ATPG in Compression mode

---

1. Run the TetraMAX command file ./tmax/compression.cmd to run ATPG in Adaptive Scan mode:

```
cd tmax
tmax compression.cmd &
```

**Question 14.** What is the test coverage reported by TetraMAX ?

.....

**Congratulations!** This completes the DFTMAX lab.

## Answers / Solution

For help with the scripts/commands, see the **.solutions** directory.

- Question 1.** How many scan chains will be inserted, and how will this affect the number of tester cycles on the tester?
- 2 scan chains. This will make the scan chains longer, which will affect the number of tester cycles needed per pattern hence increasing the Test Data Volume and Test Time.
- Question 2.** What is one way to reduce Test Data Volume?
- Use Adaptive Scan technology to insert compression.
- Question 3.** Do you need to specify the compression ratio?
- Yes. The compression ratio defaults to 10. So you only need to specify it if you are reducing or increasing this number.
- Question 4.** What do you see now in the preview report?
- You see two modes specified. One for Internal\_scan with 5 chains and one for Compression\_mode with 30 much shorter chains. You also see a new Test Mode Controller Information section.
- Question 5.** Why was there a test\_modea pin added to the design?
- A control signal is need to select between the two modes created by Adaptive Scan, Internal\_scan and ScanCompression\_mode.
- Question 6.** Can you share the existing test\_mode pin that might be selecting Test Clocks?
- No. The test\_mode port was specified as a Constant type signal. It could be a signal used to fixed resets or clocks. We need a dedicated Test Mode pin for Adpative Scan.
- Question 7.** Which pins are used for scan in, scan out and scan enable?
- Same pins as specified for Normal scan
- ScanDataIn Ports:
- pad[0], pad[1], pad[2], pad[3], pad[4]
- ScanDataOut Ports:
- sd\_A[0], sd\_A[1], sd\_A[2], sd\_A[3], sd\_A[4]

ScanEnable Port:  
scan\_en

**Question 8.** Which port is now being used to select between modes?

TM\_COMP.

**Question 9.** What new messages do you see with insert\_dft?

Architecting Scan Compression structures  
Architecting Scan Chains  
Architecting Load Compressor (version 1.3)  
Number of inputs/chains/internal modes = 5/30/4  
Architecting Load Decompressor (version 1.3)  
Number of outputs/chains = 5/30  
Combinational initialization successfully completed.

**Question 10.** How many STIL Test Protocol files do you need to write out with an Adaptive Scan flow?

Two Test Protocol files, one for normal scan, and one for adaptive scan.

**Question 11.** How many netlists are needed?

Only one.

**Question 12.** What are the differences that you see?

There are new scan compression violations:

517 SCAN COMPRESSION VIOLATIONS

150 X on chain affects observe ability of other chains violations (R11)

367 X source gate is propagatable to other scancells violations (R14)

**Question 13.** Look at both test protocol files, do you see a difference?

Visually inspect the files to see added sections for Adaptive Scan.

**Question 14.** What is the test coverage reported by TetraMAX?

99.56%

# 11

## Clock Gating with Power Compiler

### Learning Objectives

After completing this lab, you should be able to:

- Understand how to insert the ICGs (Integrated clock gating cells) in your design.
- Hookup the scan\_enable or testmode ports to the top level
- Make the dft\_drc understand the clock gating circuitry



**Lab Duration:**  
**30 minutes**

## Lab 11

### Overview

#### During this lab you will:

1. Read an RTL design and insert clock gating cells.
2. Run the test DRC and insert scan.

#### Flow:

1. Analyze the RTL design into DFT.
2. Declare the clock gating style appropriately.
3. Elaborate the design to see the potential sites for clock gating.
4. Insert the clock gating.
5. hookup the test ports to the top level.
6. Create and save a test protocol; verify that the design and the test protocol are compatible.
7. Compile and save the gate-level design.
8. Remove the designs and read the mapped design and test protocol.
9. Specify scan constraints and preview the scan architecture that will result from applying them.
10. Insert the scan chain.
11. Hand off the design and related files for use by downstream tools; exit.

## File Locations

All files for this lab except for designs and libraries are located in the directory **lab11\_power**.

### Directory Structure

|                           |                                 |
|---------------------------|---------------------------------|
| <b>lab11_power</b>        | Current working directory       |
| analyzed                  | Intermediate acs_read_hdl files |
| logs                      | Session log files               |
| unmapped                  | Unmapped protocol               |
| mapped                    | Gate-level netlist              |
| mapped_scan               | Gate-level scan design netlist  |
| reports                   | DFTC reports                    |
| tmax                      | Files for downstream tools      |
| scripts                   | Constraint and run scripts      |
| ../ref/rtl/RISC_CORE/vhdl | Design files                    |
| ../ref/db                 | Library files                   |

## Lab 11

### Instructions

Your goal is to modify and run the scripts to declare the clock gating style and elaborate the design. Then insert the clock gating cells and hookup the scan ports. After this point, define test protocol, preview the scan architecture, insert scan chains.

The scripts are:

```
1a_read_design.tcl
1b_insert_clock_gating.tcl
2create_test_protocol.tcl
3compile_and_save.tcl
4read_gate_and_protocol.tcl
5preview_dft.tcl
 settings_insert_dft.tcl
6insert_dft.tcl
7handoff.tcl
```

### Task 1. Read RTL Design and Insert Clock Gating

1. Invoke DFTC.

```
cd lab11_power
design_vision
```

2. Examine the script used for reading the RTL, then execute it

```
exec cat scripts/1a_read_design.tcl
s scripts/1a_read_design.tcl
```

**Note:** Notice the use of the “-no\_elaborate” switch. This prevents RTL elaboration, which is necessary if you are going to use automatic clock gating!



## 3. Modify and execute the script `scripts/1b_insert_clock_gating.tcl`:

- a. Insert clock gating logic to add a control point *after* the clock gating cell and use `scan_enable` for the control logic.

**Note:** This library contains only integrated gating cells with post-control, that is why you have to set the control point to *after*. To use integrated gating cells, the options “-positive\_edge\_logic {integrated}” and “-negative\_edge\_logic {integrated}” were used.

**Question 1.** What are the added options?

.....

- b. Use the correct command so that the tool hooks up the top level `scan_enable` signal to the clock gating control logic.

**Question 2.** What is the command?

.....

## 4. Check the clock gating logic inserted using:

```
report_clock_gating
```

**Question 3.** How many clock gating elements were added?

.....

**Question 4.** Did the tool create a `test_se` port at the top level? Why?

.....

## Lab 11

### Task 2. Create the test protocol, compile the design and re-read the gate level netlist

---

1. Create the test protocol at the RTL level and run dft\_drc:

```
s scripts/2create_test_protocol.tcl
```

**Question 5.** Do you see any RTL dft\_drc violations?

.....

2. Compile the design:

```
s scripts/3compile_and_save.tcl
```

**Note:** During compile, notice the “Mapping integrated clock gating circuitry” statements.

3. Review the resulting schematic, which was generated after 1 pass compile, in the GUI (Menu: *Schematic* → *New Design Schematic View*). Use the highlighting tools to trace the TEST\_SE signal. The integrated clock gating cells in the library are named after the pattern gclrsn\* and gclfsn\*.

**Question 6.** Do you see top level TEST\_SE connections to clock gating control cells?

.....

4. Re-Read the gate level netlist and the test protocol:

```
s scripts/4read_gate_and_protocol.tcl
```

**Question 7.** How many scan cells have violations?

.....

**Question 8.** What scan attributes are there on the TEST\_SE pin?

.....

5. Check this by reporting which dft signals are currently defined:

```
report_dft_signal -view exist
report_dft_signal -view spec
```

## Task 3. Modify test protocol, preview, insert scan

1. Modify the `scripts/settings_insert_dft.tcl` script to specify the correct constraints for top level scan enable. After your modifications, execute `scripts/5preview_dft.tcl`, which sources the settings.

**Question 9.** How many cells have violations now?

.....

2. Display the default scan architecture that will be implemented for this design.

```
preview_dft
```

**Question 10.** How many scan chains will be inserted?

.....

**Question 11.** Will scan insertion use existing functional pins as the scan signals (scan in, scan out and scan enable for each scan chain) or will it create new ones (test\_si, test\_so, test\_se ports)?

.....

3. Once there are no scan violations, insert the scan and run the hand off steps:

```
s scripts/6insert_dft.tcl
```

**Question 12.** What is the estimated test coverage?

.....

```
s scripts/7handoff.tcl
```

## Lab 11

### Task 4. **OPTIONAL: Repeat the entire lab using TEST\_MODE as the control signal**

---

1. Modify the script **1b\_insert\_clock\_gating.tcl** to specify the TEST\_MODE signal instead of TEST\_SE.

**Note:** This information has to be specified in 2 commands! If you get stuck, have a look at the .solutions directory.

2. Verify the results of dft\_drc at the RTL and gate level.

**Question 13.** What is the difference here?

.....  
.....

3. Compare the coverage results in this case to the answer in Question 12.

**Question 14.** How do you explain the differences?

.....  
.....

**Congratulations!** This completes the DFT Power Compiler lab.

## Answers / Solutions

**Question 1.** What are the added options?

```
set_clock_gating_style \
-sequential_cell latch \
-positive_edge_logic {integrated} \
-negative_edge_logic {integrated} \
-minimum_bitwidth 4 -max_fanout 32 \
-control_point after \
-control_signal scan_enable
```

**Question 2.** What is the command?

See `.solutions/1b_insert_clock_gating.tcl`

```
hookup_testports -se_port TEST_SE
```

**Question 3.** How many clock gating elements were added?

37

**Question 4.** Did the tool create a test\_se port at the top level? Why?

No. TEST\_SE is an existing port and was specified as se port in question 2.

**Question 5.** Do you see any RTL dft\_drc violations?

There should be no violations.

**Question 6.** Do you see top level TEST\_SE connections to clock gating control cells?

Yes. Highlight the nets connected to the TEST\_SE port (ctrl-e), and trace them inside the blocks. You should see connections to blocks named similar to clk\_gate\_\*.

**Question 7.** How many scan cells have violations?

329 out of 349 cells:

SEQUENTIAL CELLS WITH VIOLATIONS

- \* 292 cells have test design rule violations
- \* 37 cells are clock gating cells

SEQUENTIAL CELLS WITHOUT VIOLATIONS

- \* 20 cells are valid scan cells

**Question 8.** What scan attributes are there on the TEST\_SE pin?

None.

### **Task 3, step 1:**

See `.solutions/settings_insert_dft.tcl`

The following lines need to be added to `scripts/settings_insert_dft.tcl` :

```
set_dft_signal -view spec -port TEST_SE \
 -type ScanEnable -active_state 1
set_dft_signal -view existing_dft -type ScanEnable \
 -active_state 1 -port TEST_SE
```

**Question 9.** How many cells have violations now?

37 cells (which are the clock gating cells). 312 cells are valid scan cells.

**Question 10.** How many scan chains will be inserted?

1.

**Question 11.** Will scan insertion use existing functional pins as the scan signals (scan in, scan out and scan enable for each scan chain) or will it create new ones (test\_si, test\_so, test\_se ports)?

DFTC will use existing functional pins as the scan signals.

**Question 12.** What is the estimated test coverage?

99.69%

**Question 13.** What is the difference here?

There are 0 violations for RTL and at the gate-level dft\_drc checks, while you had many more violations earlier at the gate-level.

**Question 14.** How do you explain the differences?

The coverage is now only 95.62%. If you remember the lecture, it is expected that test coverage will be lower when using test\_mode, since the registers controlling the gating cells are not tested.

This page was intentionally left blank.